Data Centric AI

DATA PREPARATION

Business Intelligence Group

Matteo Golfarelli

Data Preparation

Data preparation is aimed at making data ready to be submitted to the model. It involves many activities such as:

1. Data encoding
2. Data cleaning
3. Dataset Completion, Resampling & Data Augmentation
- Dataset deduplication — we didn't talk it in detail
- *Feature selection*
- *Feature engineering*

they have a section in particular even if they are in Data prep phase

Encoding Categorical Data

Encoding is the process of transforming data from its original format (often categorical, textual, or symbolic) into a numerical representation that can be processed by machine learning models.

- Encoding makes data understandable to machines — same algo can run directly on categorical data
- Poor encoding can introduce bias or false relationships between features

is more complex set of technics (so, Encoding and Embedding are not synonymous), like T-SNE and PCA

An **embedding** is a mapping of discrete or high-dimensional data (such as words, images, or entities) into a continuous, low-dimensional vector space (latent), where similar items are represented by vectors that are close to each other according to a chosen distance metric

- All embeddings are encodings, but not all encodings are embeddings
- Encoding is typically rule-based while embeddings are learnt from data — Embedding is learnt from the data (e.g T-SNE)
- Embeddings turn meaning into geometry
- Embeddings capture hidden structure in data representing it through a **latent space**: a hidden vector space where data is represented in a compact way where the dimensions are not directly interpretable (they are not “weight”, “height”, etc.)

Encoding Nominal Attributes: One-hot encoding

Type of feature without ordering properties (only distinction)

- The representation of a discrete attribute by a set of binary attributes is called binarization

Label Encoding

Eye color	Encoding
Black	0
Brown	1
Azure	2
Green	3
Gray	4

Binary Encoding

X1	X2	X3
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0

This naive encodings translate each category in a integer/set of bits and can lead the ML technique to: (queste relazioni non esistono ma vengono apprese dal modello a causa della tecnica di encoding)

- infer a relationship between "Azure" and "Green" since both have the bit X2=1
- consider "Black" and "Brown" more similar than "Black" and "Green"
- consider "Gray" better/worse than "Black"

With **one-hot encoding** each category becomes a binary column (0 or 1)

- In each row, only the column corresponding to the present category has a value of 1
- This prevents assigning an arbitrary numeric order to categories (which would happen with label encoding) there is no relationship between values (so, more robust than the two naive encodings)
- Is not appropriate for high cardinality attributes since determines a sparse representation (i.e. it requires more bits than the naïve label encoding) (one column for each value)

Eye color	X1	X2	X3	X4	X5
Black	1	0	0	0	0
Brown	0	1	0	0	0
Azure	0	0	1	0	0
Green	0	0	0	1	0
Gray	0	0	0	0	1

linear equal distance

Encoding Ordinal Attributes: Ordinal encoding

Each category is mapped to an integer that respects the natural order of the attribute

Satisfaction level	Encoding
Low	1
Medium	2
High	3



While it works well with many models (especially tree-based ones)

- The model assumes equal spacing between values (same distance between values)
  - Distance between “Low” and “Medium” = “Medium” and “High”
- May distort the results of techniques that use distance metrics (e.g. k-means)
- Non-linear distance can be manually imposed or learned through **embeddings**
  - Need to choose when there is no equal distance between all values

Label vs Ordinal Encoding: A Visual Example

Code: [Synthetic_Encoding.ipynb]

Generate a 2D dataset with 2 clusters of data

Transform numerical attributes to ordinal ones through binning
VL, L, M, H, VH

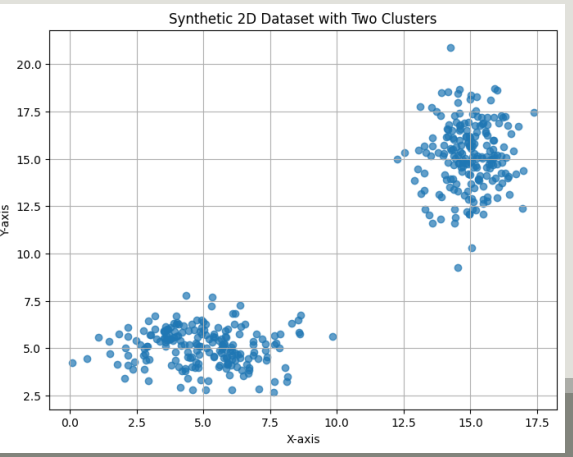
Distance information is not preserved

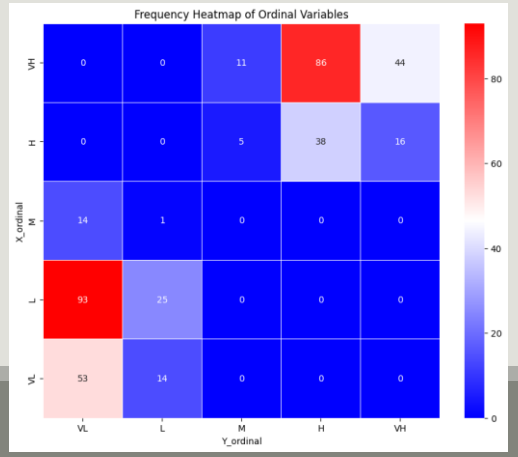
Apply Label encoding
(doesn't represent order)

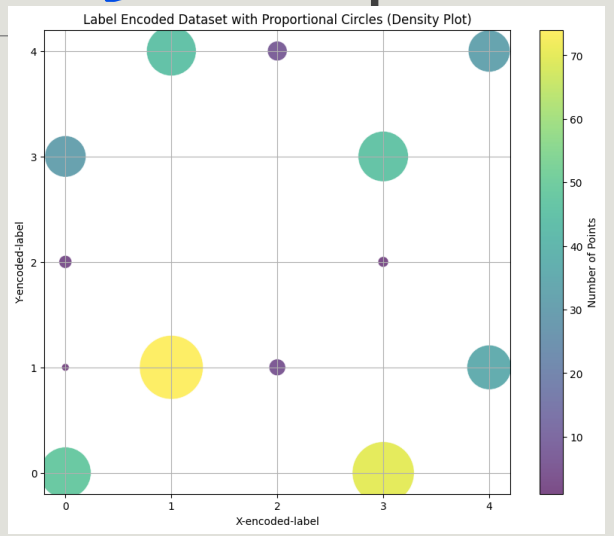
Apply Ordinal encoding

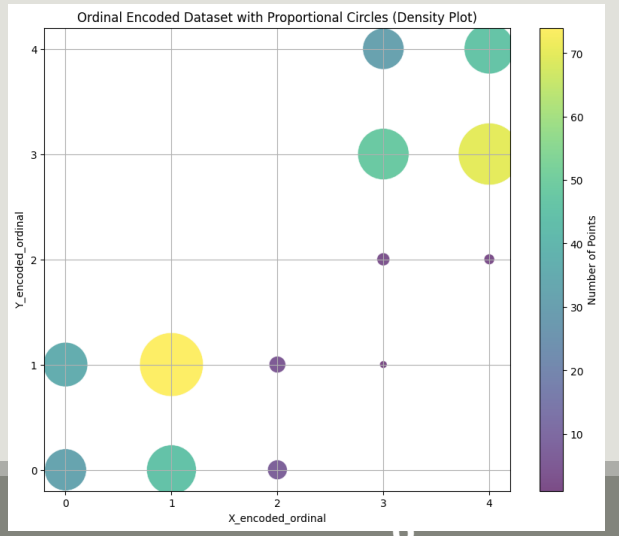
Distance information is preserved

(distance between bins is equal)









not equal distance

Embedding Ordinal Attributes

Distance-preserving ordinal embedding can be learned through a target variable expressing what is close or distant (e.g. customer satisfacion is strictly related to their spending: spend a lot, they are satisfied and viceversa)

Given a dataset of customers, we want to predict their monthly spending based on satisfaction (low, medium, high).

Customer	Satisfaction	Spending
1	low	50
2	medium	120
3	high	200
4	medium	130
5	high	210

We could infer a 2-dimensional latent space where *medium* is closer to *low* rather than to *high*

- Dist(low, medium)= 0.485
- Dist(medium, high)= 0.516

Here, we have a non-linear distance between values

Satisfaction	embedding_vector
low	[0.0, 0.0]
medium	[0.4839, 0.0323]
high	[1, 0.0323]

Target Encoding

Target encoding maps each category *c* to a statistic of the target variable *y* — usually the **mean** — computed from all training instances belonging to that category

- For each category c: Encoded(*c*) = mean(*y* | attribute = *c*)

Encoded("Medium") = mean of 120+130 (che sono i valori di MonSpend quando Satis è Medium). Quindi attribuiamo il valore numerico 125 a Medium

The **target variable** is **any numeric variable** that captures the ordering of the categories — it does NOT have to be the class label of the ML task; it could be a rating, a duration, or a spending amount

- To encode "Satisfaction" (Low/Med/High), use "Monthly Spending" as target — a variable that reflects how much each satisfaction level is associated with customer value

Advantages

- Naturally captures the relationship between the categorical attribute and the target variable
- Works well with **high-cardinality** attributes where one-hot encoding would explode dimensionality
- Produces a single compact numeric column

Target Encoding: A Worked Example

Encoding customer **Satisfaction** (Low / Medium / High) using **Monthly Spending** as target variable

- Monthly Spending is chosen because it reflects how much each satisfaction level is associated with customer value — it is not the class label of the task

Satisfaction	Count	Mean Spending
Low	3	60
Medium	5	120
High	4	200

Target Encoding: A Worked Example

Encoding customer **Satisfaction** (Low / Medium / High) using **Monthly Spending** as target variable

- Monthly Spending is chosen because it reflects how much each satisfaction level is associated with customer value — it is not the class label of the task
- It could violate monotonic property of the ordering that could be restored through Isotonic regression

Satisfaction	Count	Mean Spending
Low	3	60
Medium	5	120
High	4	200
Very High	2	180

Per evitare che per certi valori dell'attributo ordinale la relazione con il target variable non venga rispettato, aggiungiamo questa tecnica per mantenere la proprietà invartiata: in questo esempio, High ha un encoding numerico maggiore rispetto a very high, nonostante very high è maggiore di high nell'ordinamento

Isotonic Regression

il concetto di monotonia (o di ordine monotono) si riferisce a una relazione che mantiene l'ordinamento.

Isotonic regression is a non-parametric supervised technique that fits the best piecewise constant, monotone (non-decreasing or non-increasing) function to a set of ordered observations

- Suppose we observe pairs (x_i, y_i). Isotonic regression finds a function $f()$ that minimizes squared prediction error:

$$min_f \sum_i (y_i - f(x_i))^2 \qquad \text{subject to } x_i < x_j \rightarrow f(x_i) \leqslant f(x_j)$$

Non entriamo in dettaglio sull'algo usato per calcolarlo

- Solved by the **Pool Adjacent Violators (PAV)** algorithm in O(n) time: whenever two adjacent values violate the monotone order they are merged and replaced by their weighted average

Isotonic Regression for Ordinal Encoding

Ordinal encoding assumes equal spacing between categories; **isotonic regression** learns the optimal spacing that respects the monotone order while fitting a numeric target variable

- Assigns integer ranks to categories (e.g., Low=1, Med=2, High=3) and fits isotonic regression on those ranks vs. the target variable
- The fitted values become the **learned encoding** — distances between categories reflect the actual relationship with the target

Advantages over linear ordinal encoding

- Captures **non-linear** inter-category relationships driven by the data
- Guarantees monotonicity: the learned order always respects the original ordinal structure
- Unlike target encoding, the PAV algorithm **automatically corrects violations** — it merges categories whose means break the order, rather than simply reporting them

Limitation

- Requires a numeric target variable; in purely unsupervised contexts a proxy target or domain knowledge is needed

Target Encoding & Isotonic Regression: A Worked Example

Running example: encoding customer **Satisfaction** (Low / Medium / High) using **Monthly Spending** as target variable

- Monthly Spending is chosen because it reflects how much each satisfaction level is associated with customer value — it is not the class label of the task

Target encoding: replace each category with the mean spending of instances in that category

Isotonic regression: when a category's mean **violates** the order (e.g., High mean < Medium mean), isotonic regression pools the violating categories and replaces both with their weighted mean (Pool Adjacent Violators algorithm)

the system search for the best value that respect the ordering

Satisfaction	Count	Mean Spending
Low	3	60
Medium	5	120
High	4	200
Very High	2	180

Satisfaction	Target Enc.	Isotonic
Low	60	60
Medium	120	120
High	200	193
Very High	170	193

Car Evaluation

Example

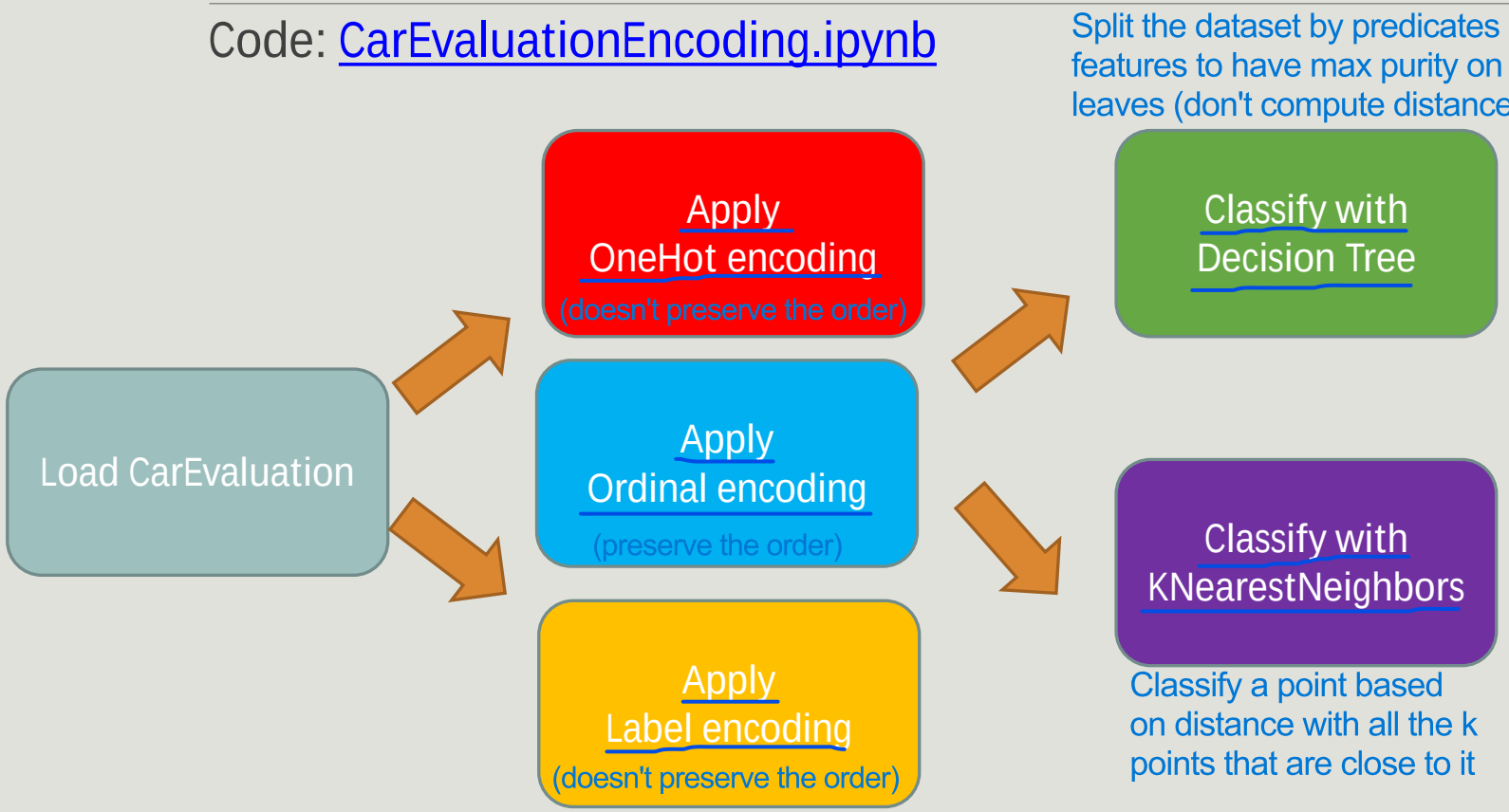
- CarEvaluation.csv dataset
  - ✓ 1728 instances
  - ✓ No missing data

All features are ordinal attributes (we need to transform them into numerical)

Attribute	Description
buying	Buying price of the car (vhigh, high, med, low)
maint	Maintenance cost of the car (vhigh, high, med, low)
doors	Number of doors (2, 3, 4, 5more)
persons	Seating capacity (2, 4, more)
lug_boot	Luggage boot size (small, med, big)
safety	Safety level (low, med, high)
class	Overall car evaluation (unacc, acc, good, vgood)

OneHot vs Ordinal Encoding

Code: [CarEvaluationEncoding.ipynb]



	Decision Tree	KNN
OneHot	Acc: 0.961 Prec: 0.932 Rec: 0.927	Acc: 0.898 Prec: 0.894 **Rec: 0.650**
Ordinal	Acc: 0.983 Prec: 0.967 Rec: 0.974	Acc: 0.944 Prec: 0.924 **Rec: 0.855**
Label	Acc: 0.979 Prec: 0.961 Rec: 0.976	Acc: 0.898 Prec: 0.905 **Rec: 0.691**

Take away: Encoding the natural order of the attributes improves every model. Distance-based models suffer most from a wrong encoding: KNN recall goes from 0.65 (one-hot) to 0.86 (ordinal), while the Decision Tree is almost unaffected."

	Forecasted values		
Actual values		Positive	Negative
	Positive	TP	FN
	Negative	FP	TN

Accuracy: the proportion of all predictions that are correct.

$$Acc = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision: among cases predicted positive, the proportion that are truly positive.

$$Prec = \frac{TP}{TP + FP}$$

Recall: among all truly positive cases, the proportion the model correctly identifies.

$$Rec = \frac{TP}{TP + FN}$$

oding

Recall is the most affected on KNN

Classify with Decision Tree

Classify with earestNeighbors

	Decision Tree	KNN
OneHot	Acc: 0.961 Prec: 0.932 Rec: 0.927	Acc: 0.898 Prec: 0.894 **Rec: 0.650**
Ordinal	Acc: 0.983 Prec: 0.967 Rec: 0.974	Acc: 0.944 Prec: 0.924 **Rec: 0.855**
Label	Acc: 0.979 Prec: 0.961 Rec: 0.976	Acc: 0.898 Prec: 0.905 **Rec: 0.691**

Take …mproves every model. Distance-based models suffer most from a wrong encoding: KNN recall goes from 0.65 (one-hot) to 0.86 (ordinal), while the Decision Tree is almost unaffected."

2

Data Cleaning

Data cleaning involves several dimensions:

Accuracy is the degree to which data correctly and sufficiently precisely represents the real-world value for the intended use.

- A thermometer with precision of ±3°C cannot be used to measure human diseases
- A patient's weight is recorded as 85 kg instead of the actual 65 kg due to a data entry error

Completeness measures whether all required data is present

- Missing values in critical features such as income in a credit scoring dataset

Consistency ensures that data does not contain contradictions across different datasets or within the same dataset

- A customer marked as "inactive" in the master data while she is actually submitting orders

Data Cleaning

Data cleaning involves several dimensions:

Uniqueness means that each real-world entity is represented only once in the dataset.

- Duplicate customer records with slightly different names

Timeliness refers to how up-to-date the data is relative to the task

- Profiling customers based on their purchase data from a year ago may lead to assumptions about behaviors that have changed in the meantime (consider the change of behaviour in time) (e.g. covid and petrol crisis)

Semantic Validity ensures that data values conform to the meaning and rules of the domain.

- A “date of birth” occurring after the “date of registration.”

A good model cannot compensate wrong data

Missing Data

Try to reduce the impact of this problem (cannot completely solve it, if we have missing data)

Key problems caused by missing data:

- Loss of statistical power — the data overall is not representative of the whole population
- Bias in model training — model learn behaviour that is not real
- Incompatibility with many ML algorithms — many ML models cannot work on missing data
- Hidden data leakage risks (if handled improperly)

Missing Data

Key problems caused by missing data:

- Loss of statistical power
- Bias in model training
- Incompatibility with many ML algorithms
- Hidden data leakage risks (if handled improperly)

Missing data can **systematically distort what the model learns** if the missingness is not random. The model doesn't just "lose data"! It can **learn a distorted version of reality**

These cannot work if we have missing data

- Linear Regression
- k-Nearest Neighbors (k-NN)
- k-Means clustering
- SVM (classification and regression)

If you fill missing values using the mean computed over the **entire dataset** including the test set!

• Model indirectly "sees the future"

Types of Missingness

MCAR (Missing Completely At Random) Missingness is purely random and unrelated to any variable.

- *Data is missing by accident*
- A sensor randomly fails during data collection. Failings are not related to other events (e.g. temperature)

MAR (Missing At Random) Missingness depends on observed variables (that are present in the dataset), but not on the missing value itself.

- *We can explain missingness using what we already know*
- INCOME is missing more often for younger patients (observed variable: AGE). But given AGE, missingness does not depend on INCOME itself.

MNAR (Missing Not At Random) (The more complex to resolve) Missingness depends on the missing value itself

- *The value is missing because of what it is* (hard because we cannot use relationship with other features)
- High-income individuals are less likely to report INCOME so missingness depends on the actual (unobserved) INCOME
- To restore ~~NMAR~~ MNAR you need to input assumptions (need domain knowledge) about the missingness mechanism

Strategies for Missingness Handling

From the simplest to the most complex technics

Delete missing points (drop rows) or attributes (drop columns)

Se ho molte righe nel dataset e ho pochi missing value di quel attributo, posso decidere di eliminare quelle righe dal dataset (la rappresentatività di quel dataset rimane pressochè la stessa). Se ci sono molti missing values per un dato attributo, meglio eliminare tutta la colonna perchè quel valore ha poco valore

Constant imputation replace with fixed value (e.g., 0 or "unknown")

- Useful when “missingness” carries meaning

quando non c'è quel dato, vuol dire che significa qualcosa

Statistical imputation substitute the missing values with:

- **Mean/Median for numeric attributes, use Mode for categorical attributes**
  - Fast, deterministic but they underestimates variance, introduces bias

(reduce variance) affect data distribution, so

- **Regression Imputation** learn a (simple) model for each variable using the the variable itself only

(model based on just one variable)

  - Can underfit complex relationships
- **Forward fill** replaces a missing value with the **most recent non-missing value** that precedes it

(ha senso se è una serie temporale e prende l'ultimo valore mandato e le condizioni/attributi rimangono su (quasi) stesso valore, ad esempio da un sensore)

Good for MAR type of missingness (not for the MNAR)

Machine Learning-Based imputation

forecast the value of the missing feature (learning a model for each variable using the other features)

- **k-Nearest Neighbors (KNN) Imputation** Fill missing value using similar instances
  - Captures local structure but computationally expensive
- **Random Forest, Neural Network, Autoencoder** learn a model for each variable using all the features
  - Captures complex patterns, but less interpretable and with higher computational cost

Side Effects of Imputation

Imputation does not solve the problem of missing data, but it can alleviate it

- Imputation is not just "filling gaps" it is **injecting assumptions** into your data
- Every method implies a belief:
  - Mean imputation → "missing values are average"
  - Forward fill → "things don't change much during time"
  - k-NN → "similar samples behave similarly"

If the assumption is wrong → bias

Side Effects of Imputation: Biasing the distribution

- Simple methods like mean/median imputation reduce variance and distort the original distribution.
  - [20k, 25k, 30k, 35k, NaN] -> [20k, 25k, 30k, 35k, 27.5k]
  - You artificially add *mass* at the mean. The dataset looks more "stable" than it really is. Variance is underestimated. i.e. whatever the NaN value it would determine a ≥ variance

(qualsiasi sia il valore, avrebbe determinato una varianza maggiore rispetto ad avere un valore uguale alla media, che contribuisce in modo nullo alla varianza)

- Distorting correlations between variables
  - True relationship: *The taller you are, the more you weigh*
  - But missing weights are replaced with the mean: Tall person → missing → replaced with average weight
  - Tall people appear to have "average" weight and the correlation between *height* and *weight* is artificially reduced
- Ignoring the missingness mechanism
  - True mechanism: *People with very high income* ***choose not to disclose it.*** (MNAR missingness)
  - If you impute with the mean, you systematically **underestimate high-income individuals**

Missing Data: Practical Workflow

Non c'è una soluzione universale: bisogna usare un metodo pragmatico

1. Analyze missingness

- % missing per feature
- patterns (heatmaps)

2. Classify type (MCAR/MAR/MNAR)

- Understand why data are missing

3. Select and apply candidate techniques (one or more)

4. Evaluate end-to-end with model-in-the-loop Evaluate the effects of the technics

- Cross-validation

5. Monitor stability to see if something change during time

Missing Data: Practical Workflow

1. Analyze missingness

- % missing per feature
- patterns (heatmaps)

2. Classify type (MCAR/MAR/MNAR)

- Understand why data are missing

3. Select and apply candidate techniques

4. Evaluate end-to-end with model-in-the-loop

- Cross-validation

5. Monitor stability

When we carry out an iterative technic, we need to avoid data leakage (while training the system, don't provide future/test values to the system, because you will see the model overperform because he already seen that data: performances are not real because are overestimated)

REMINDER to be applied during model training and data preparation

Correct Pipeline in Forecasting Tasks to Avoid Data Leakage

Step 1 — Training phase

- Fit the imputer on training data (e.g. compute the MEAN/MODE for the training data)
- Store the imputation parameters/model

Step 2 — Validation/Test phase

- Apply the same fitted imputer
- Do NOT recompute anything! (don't modify parameters)

Step 3 — Production (new unseen data)

- Apply the same imputer before prediction

Ames Housing

Example

[ames_housing_data_NoMissing.csv]
- ✓ 2929 instances
- ✓ No missing data

[ames_housing_RandMissing.csv]
- ✓ 2929 instances
- ✓ MCAR & MAR

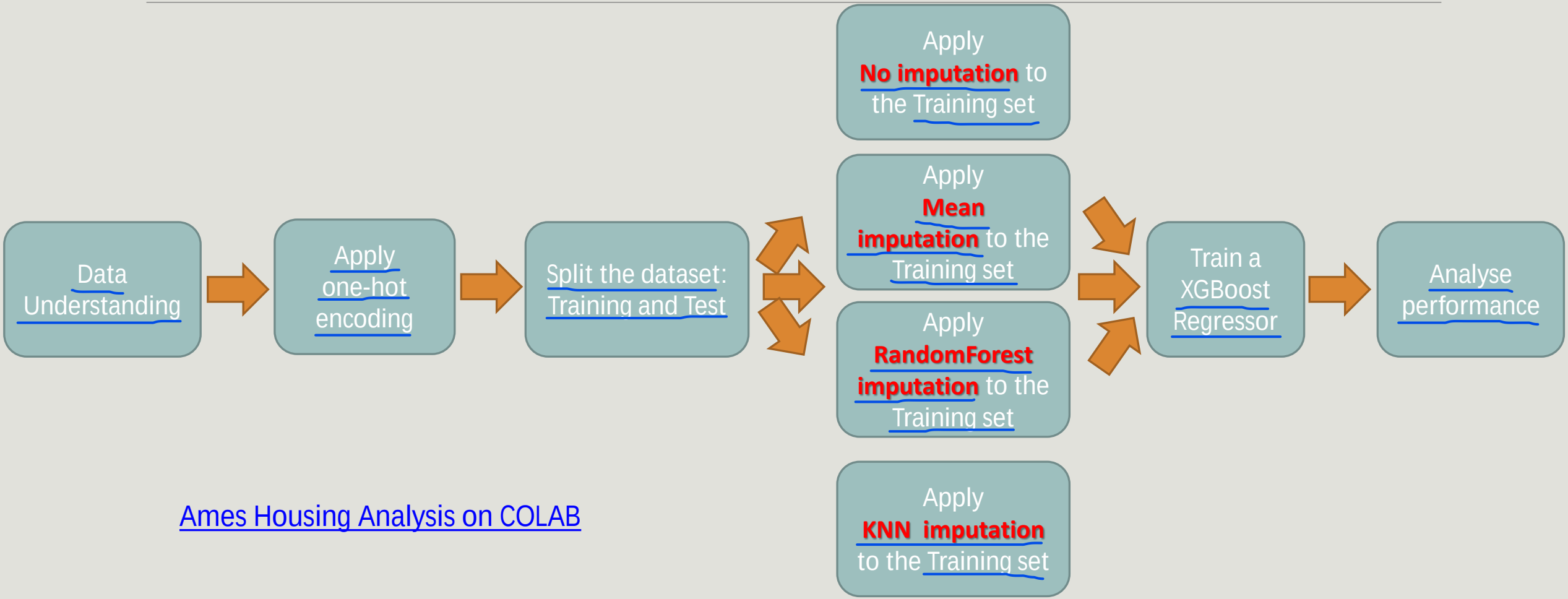
[ames_housing_NonRandMissing.csv]
- ✓ 2929 instances
- ✓ MCAR, MAR, ~~NMAR~~ MNAR

Train a regressor forecasting SalePrice

Attribute	Description
MS Zoning (Cat)	*Identifies the general zoning classification of the property*
Year Built	The year in which the house was originally constructed
Overall Qual	Rates the overall material and finish quality of the house
Overall Cond	Rates the overall condition of the house
Lot Area	Total size of the property lot in square feet
Garage Area	Size of the garage in square feet
Pool Area	Size of the swimming pool area in square feet
Bedroom AbvGr	Number of bedrooms located above ground level
Full Bath	Number of full bathrooms in the house
TotRms AbvGrd	Total number of rooms above ground level, excluding bathrooms
SalePrice	**Final sale price of the property**

Ames Housing

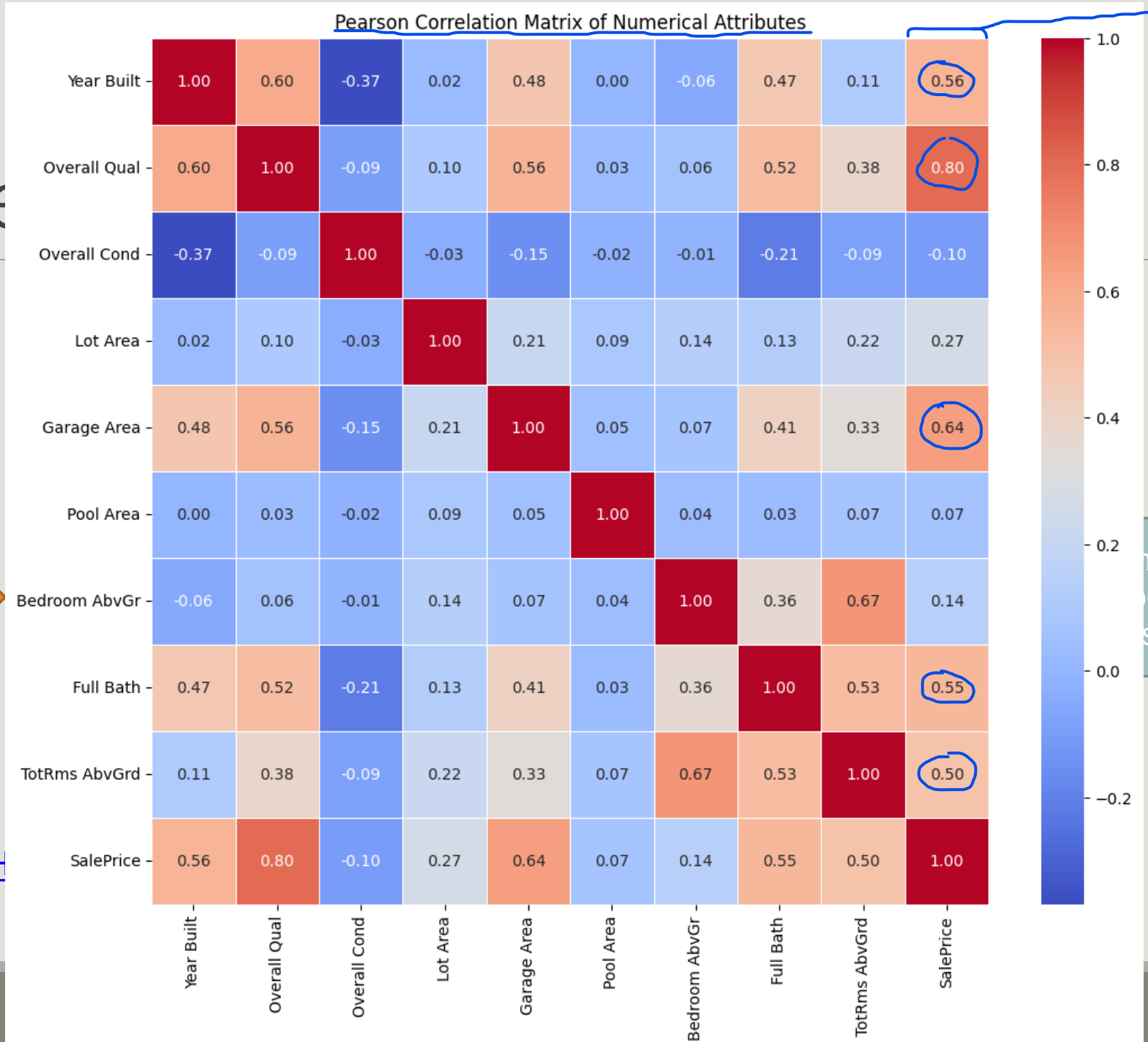
Process



[Ames Housing Analysis on COLAB]

Ame

Data Understanding

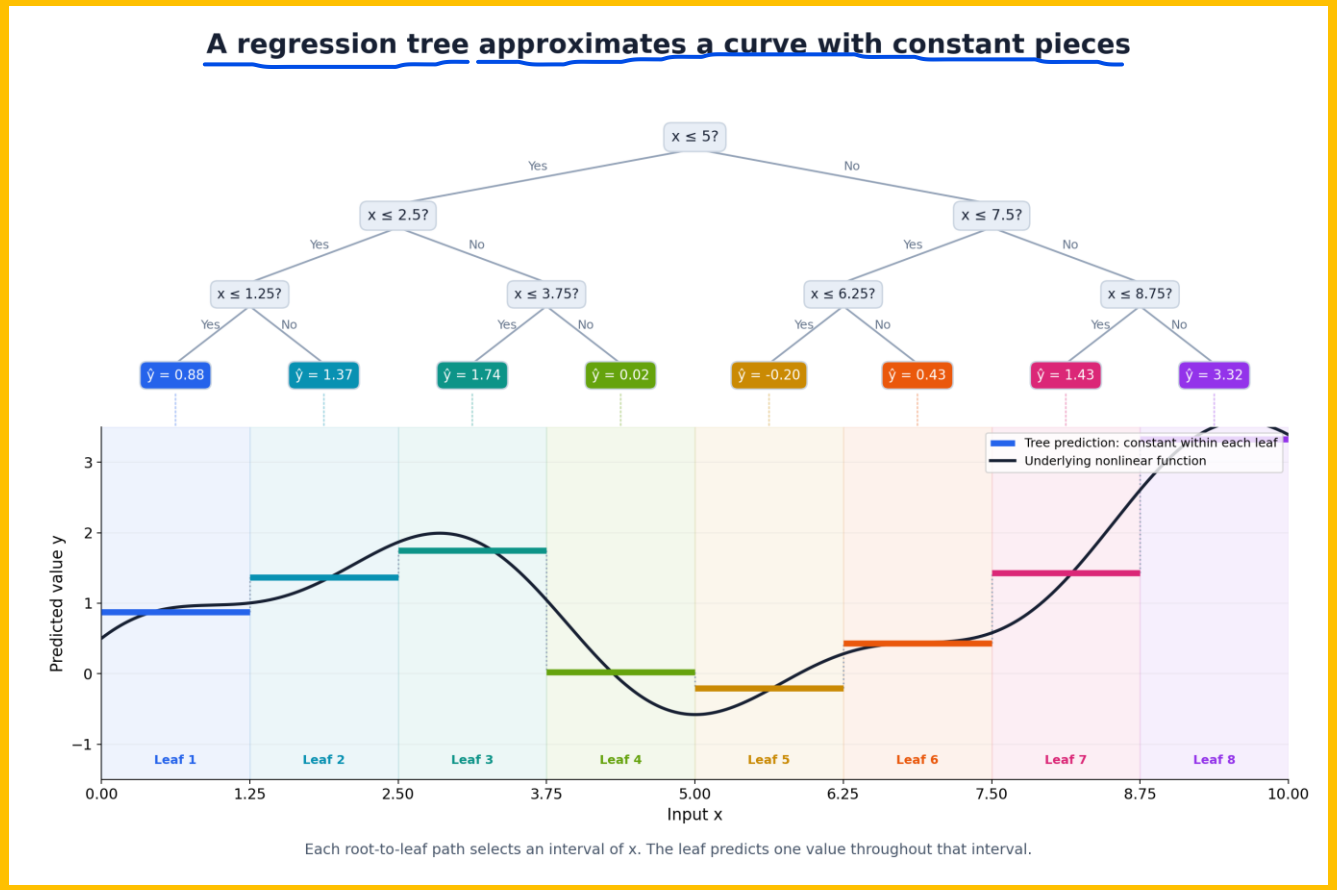


gli altri valori non cerchiati non portano informazioni importanti per predizione

Ames H

h a
ost
ssor

Analyse performance



On the leaf, you have a constant value as Regression value

[Ames Housing Analysis on COLAB]

Apply **o imputation** to he Training set

Apply **RandomForest imputation** to the Training set

Apply **KNN imputation** to the Training set

Train a XGBoost Regressor

Analyse performance

Ames Housing

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

y_i = true values $\hat{y}_i$ = predictions from XGBoost $\bar{y}$ = mean of the target values

R^2 compares the prediction error of the model vs. the error obtained by always predicting the mean target value. So R^2 tells you: "How much variance of the target is explained by the model?" **A perfect prediction implies R^2 = 1 (while R^2 = 0 same as predicting the mean)**

Generalmente, abbiamo performance più basse rispetto alle altre tecniche

Stesso valore perchè non ci sono missing values

XGBoost R-squared	No Missing	Rand Missing	Non Rand Missing
No imputation	0,848	0,800	0,838
Mean imputation	0,848	0,785	0,845
RandomForest imputation	0,848	0,867	0,880
KNN imputation	0,848	0,782	0,852

There are variables that can bring/carry info to forecast the overall quality based on other features (Pearson correlation shows it)

Ames Housing

y_i = true values $\hat{y}_i$ = predictions from XGBoost $\bar{y}_i$ = mean of the target values

R^2 ction error of r o licting

So R^2 tells y e c d by the

diction implie e as

Overall Qual

Original dataset has:
Mean: 6,10
StDev: 1,41

Overall Qual

Mean: 6,10
StDev: 1,44

Overall Qual

Aggiungendo la media ha ridotto la varianza del dataset originale sulla feature "overall quality"
Mean: 6,10
StDev: 1,20

Overall Qual

Media e StDev non cambiano
Mean: 6,09
StDev: 1,37

XGBoost R-squared	No Missing	Rand Missing	Non Rand ssing
No imputation	0,848	0,800	,838
Mean imputation	0,848	0,785	0,845
RandomForest imputation	0,848	0,867	0,88
KNN imputation	0,848	0,782	0,852

Overall Qual

Mean: 6,15
StDev: 1,36

Ames Housing

y_i = true values $\hat{y}_i$ = predictions from XGBoost $\bar{y}_i$ = mean of the target values

R^2 compares the predictio . the err predicting the mean target value. So h varian ained by the model?" A perfect predicti = 1 sam an)

Overall Qual

Mean: 6,10
StDev: 1,41

Overall Qual

Mean: 6,37
StDev: 1,34

Overall Qual

Mean: 6,37
StDev: 1,21

XGBoost R-squared	No Missing	Rand Missing	Non Rand Missing
No imputation	0,848	0,800	0,838
Mean imputation	0,848	0,785	0,845
RandomForest imputation	0,848	0,867	0,880
KNN imputation	0,848	0,782	0,852

Overall Qual

Mean: 6,18
StDev: 1,33

Overall Qual

Mean: 6,28
StDev: 1,28

Ames Housing

R^2 y_i = true values $\hat{y}$ …osty $\bar{y}_i$ = …es

…R^2 compares the pre… vs. the erro… …dicting
…he mean target valu… …uch varianc… …ed by the
…model?" A perfect pr… …R^2 = 1 same…

Overall Qual	0.606483
Full Bath	0.117389
MS Zoning_RL	0.051208
Pool Area	0.050086
MS Zoning_RM	0.034282

Overall Qual	0.351712
Garage Area	0.098879
MS Zoning_RM	0.079576
TotRms AbvGrd	0.077281
Pool Area	0.071992

Overall Qual	0.399449
Garage Area	0.082511
Pool Area	0.081507
MS Zoning_RM	0.056367
TotRms AbvGrd	0.054350

XGBoost R-squar…	No Missing	…and Missing	Non …nd Missing
No imputation	0,848	0,800	0,838
Mean imputation	0.848	0,785	0,845
RandomForest imputation		0,867	0,8…
KNN imputati…		0,782	0,852

Overall Qual	0.376903
Garage Area	0.146184
MS Zoning_RM	0.087920
TotRms AbvGrd	0.064641
MS Zoning_RL	0.058226

Overall Qual	0.568712
MS Zoning_RM	0.121994
MS Zoning_RL	0.039810
Pool Area	0.039706
Lot Area	0.037030

Ames Housing

R^2 y_i = true values $\hat{y}$ …osty $\bar{y}_i$ = …es

…R^2 compares the pre… vs. the erro… …dicting
…he mean target valu… …uch varianc… …ed by the
…model?" A perfect pr… …R^2 = 1 same…

Overall Qual	**0.606483**
Full Bath	0.117389
MS Zoning_RL	0.051208
Pool Area	0.050086
MS Zoning_RM	0.034282

Overall Qual	**0.551533**
MS Zoning_RM	0.097912
MS Zoning_RL	0.059879
Garage Area	0.046336
TotRms AbvGrd	0.045522

Overall Qual	**0.601322**
MS Zoning_RL	0.053294
Full Bath	0.053094
Garage Area	0.044164
MS Zoning_RM	0.044012

XGBoost R-squar…	No Missing	Rand Missing	Non Rand M…ng
No imputation	0,848	0,800	0,838
Mean imputation	0,848	0,785	0,845
RandomForest imputation		0,867	0,880
KNN imputation			0,852

Overall Qual	**0.530128**
MS Zoning_RM	0.093113
Full Bath	0.052587
MS Zoning_RL	0.045357
Garage Area	0.042704

Overall Qual	**0.656598**
Full Bath	0.074689
MS Zoning_RM	0.049117
Lot Area	0.033090
Bedroom AbvGr	0.032167

Scaling

Scaling the variables

If data values vary widely, objective functions will not work properly without normalization in some ML algorithms.

- Any model based on calculating distances is likely to fail

$$\text{dist}(p,q) = \sqrt{\sum_i (p_i - q_i)^2}$$

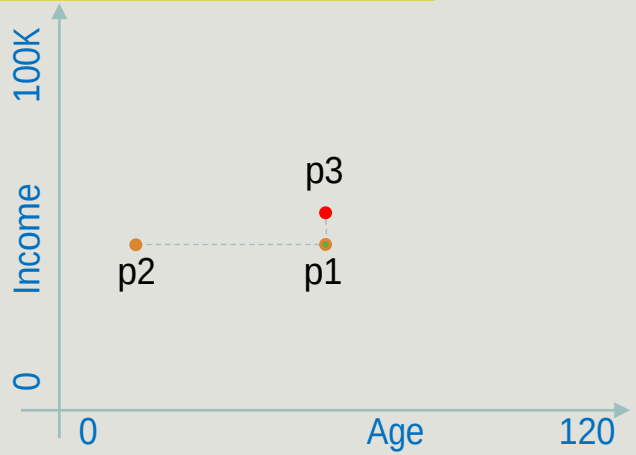
If one of the features has a broad range of values, the distance will be governed by this particular feature.

- Consider a dataset with two features age ∈[0,120] and income ∈[0,100000]
  - ✓ p1=(age = 60, income = 40,000€)
  - ✓ p2=(age = 10, income = 40,000€)
  - ✓ p3=(age = 60, income = 45,000€)

d(p1,p2)=50 **d(p1,p3)=5000**

In realtà, p1 e p2 sono più distanti rispetto a p1 e p3

age varia molto di meno rispetto ad income, quindi age ha meno impatto sul calcolo della distanza rispetto ad income



Scaling

perchè si basano principalmente sul calcolo della distanza

Which techniques are most vulnerable?

Algorithm family	Sensitivity to scaling
KNN	Very high
K-Means	Very high
SVM	Very high
PCA	Very high
Neural Networks	High
Logistic/Linear Regression	Medium–High
DBSCAN	Medium–High
Naive Bayes	Low–Medium
Decision Trees	Low
Random Forest / XGBoost	Very low



Scaling

Different scalers can be adopted:

- MinMax scaler: maps x from $[x_{min}; x_{max}]$ to $[a;b]$ that is typically [0;1]

$$x' = a + \frac{(x - x_{min})(b - a)}{x_{max} - x_{min}} \qquad x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

(underestimate the difference between values of the variabile, caused by outlier that modify min and max)

✓ MinMax scaler is sensitive to outlier and requires the bounds to be known in advance (max and min on training set; no data leakage)

- Robust scaler: Use the median (Q_2) and the interquartile range – $(Q_3 - Q_1)$ to avoid extreme values to squeeze the distribution

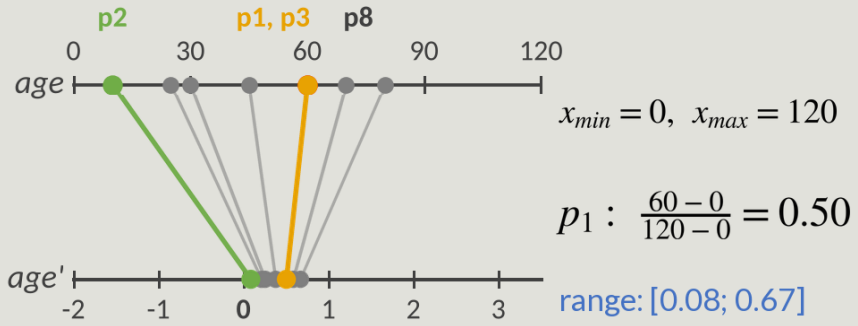
$$x' = a + \frac{x - Q_2}{Q_3 - Q_1}$$

✓ Robust scaler is not sensitive to outliers, but there is no fixed output range (cannot define a range because normalised by the Quartile not by the extremes)

Scaling

Example: 8 people, *age* ∈[0,120], *income* ∈[0,100K]; p1, p2, p3 as in the previous example, p8 earns 98,000€

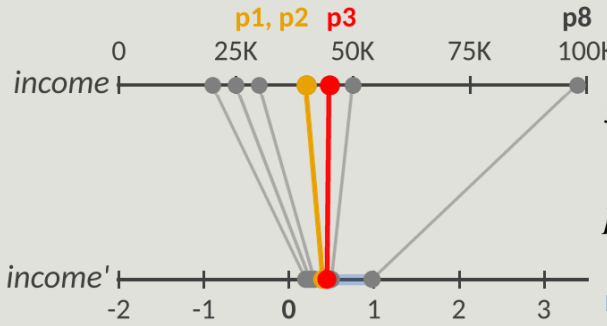
- MinMax scaler (a = 0, b = 1): using range [0,100K], p8 is mapped to 0.98; other 7 incomes fall in [0.20; 0.50]



$x_{min} = 0, \; x_{max} = 120$

$p_1 : \; \frac{60 - 0}{120 - 0} = 0.50$

range: [0.08; 0.67]

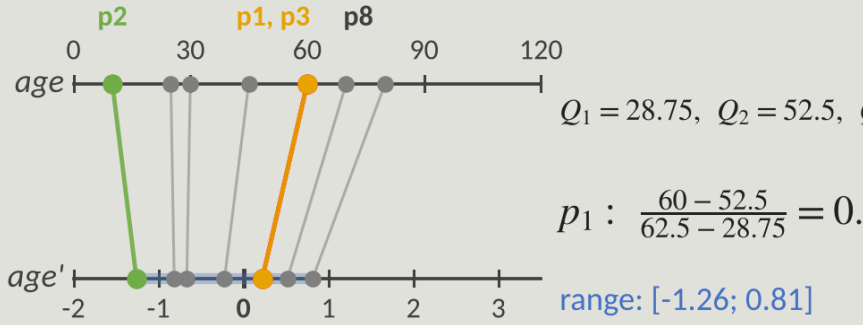


$x_{min} = 0, \; x_{max} = 100$

$p_1 : \; \frac{40 - 0}{100 - 0} = 0.40$

range: [0.20; 0.98]

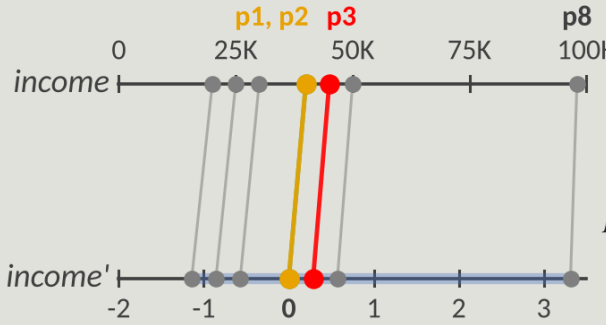
- Robust scaler (a = 0): the median is mapped to 0, the incomes keep their spread, p8 is mapped to 3.31



$Q_1 = 28.75, \; Q_2 = 52.5, \; Q_3 = 62.5$

$p_1 : \; \frac{60 - 52.5}{62.5 - 28.75} = 0.22$

range: [-1.26; 0.81]



$Q_1 = 28.75, \; Q_2 = 40, \; Q_3 = 46.25$

$p_1 : \; \frac{40 - 40}{46.25 - 28.75} = 0.00$

range: [-1.14; 3.31]

Scaling

Different scalers can be adopted:

- Z-score scaler (Standard scaler in sklearn): maps *x* to a zero mean and unit variance distribution

$$x' = \frac{x - \mu}{\delta}$$

la distanza viene misurata in multipli di StdDev

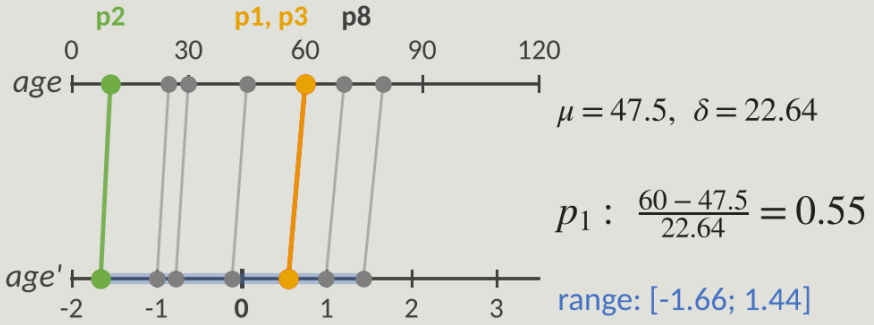
  - ✓ A z-score indicates how far a value is from the mean, *measuring that distance in standard deviations*.
    - ✓ $x' = 0$: the observation equals the mean;
    - ✓ $x' = 1.5$: the observation is 1.5 standard deviations above the mean;
  - ✓ The z-score allows you to:
    - ✓ compare variables expressed on different scales (it is dimensionless);
    - ✓ identify potentially outlier values;
    - ✓ calculate probabilities when the distribution is normal.
  - ✓ Z-Score scaler is sensitive to outlier: The mean and standard deviation are not robust statistics. Extreme observations can substantially affect both parameters and compress most standardized values around zero.

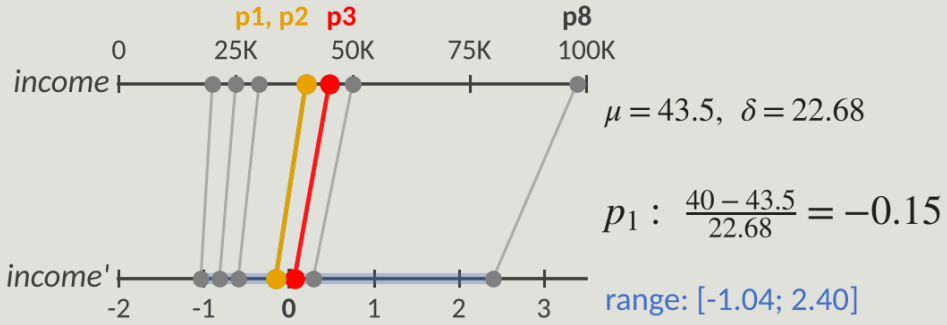
because

Scaling

Example: 8 people, *age* ∈[0,120], *income* ∈[0,100000]; p1, p2, p3 as in the previous example, p8 earns 98,000€

- Z-score scaler: p8 increases both μ and δ, so the other 7 incomes are compressed in [−1.04; 0.29]





Scaler	[0,120] → age'	[0,100000] → income'	d(p1,p2)	d(p1,p3)
No scaling	[0; 120]	[0; 100,000]	50	5,000
MinMax	[0; 1]	[0; 1]	0.42	0.05
Robust	[−1.56; 2.00]	[−2.29; 3.43]	1.48	0.29
Z-score	[−2.10; 3.20]	[−1.92; 2.49]	2.21	0.22

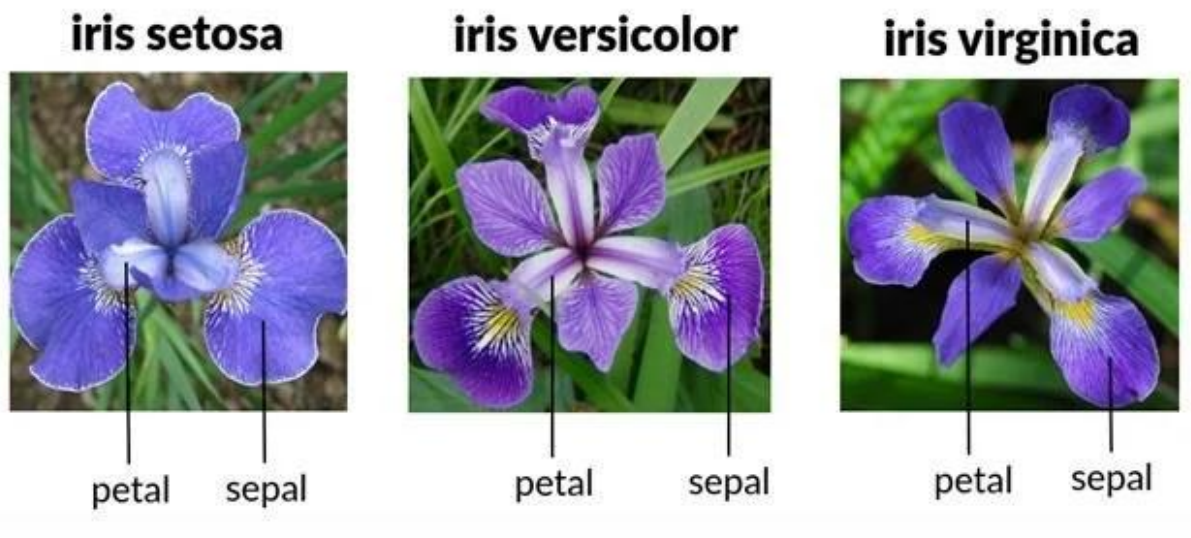
✓ Both features now contribute to the distance; with an outlier, MinMax ~~shrinks income differences the most~~ is more effected

Scaling the Iris Dataset

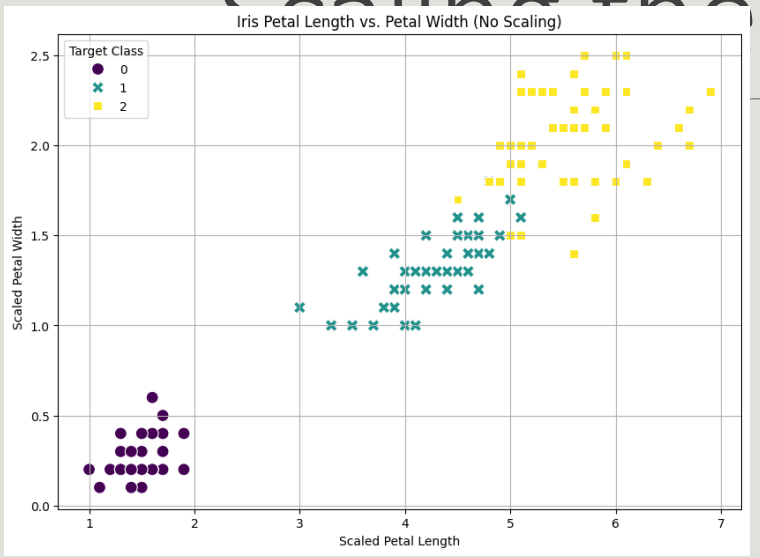
Example

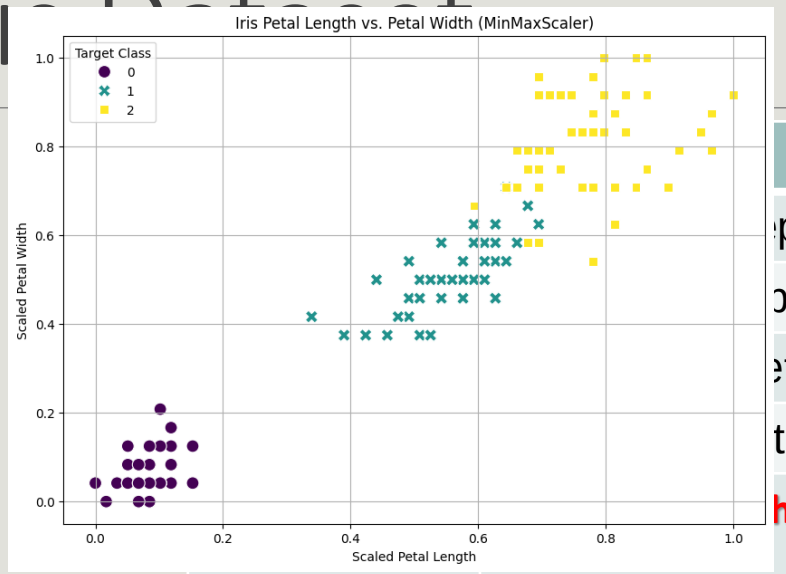
- iris dataset
  - ✓ 150 instances
  - ✓ No missing data

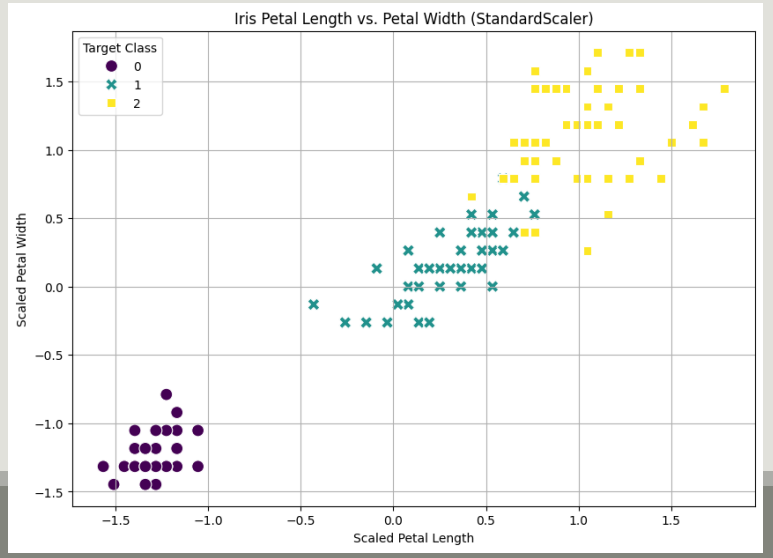
Attribute	Description
sepallength	Length of the sepal
sepalwidth	Width of the sepal
petallength	Length of the petal
petalwidth	Width of the petal
class	**The species in the Iris family**

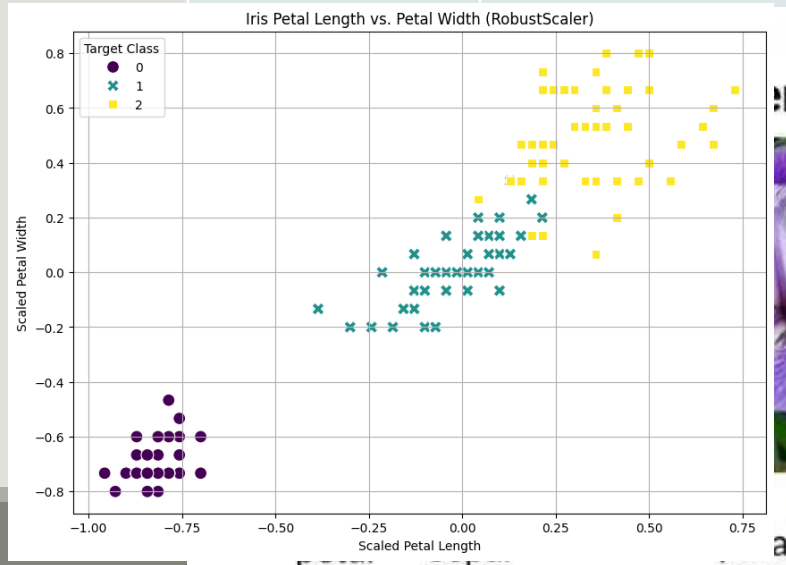


Scaling the Iris Dataset



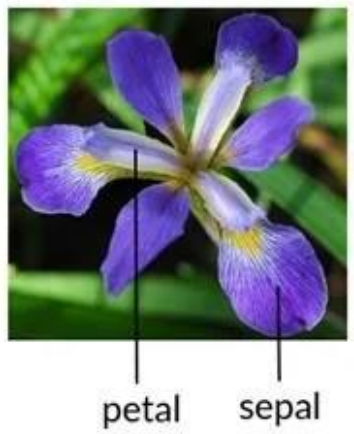






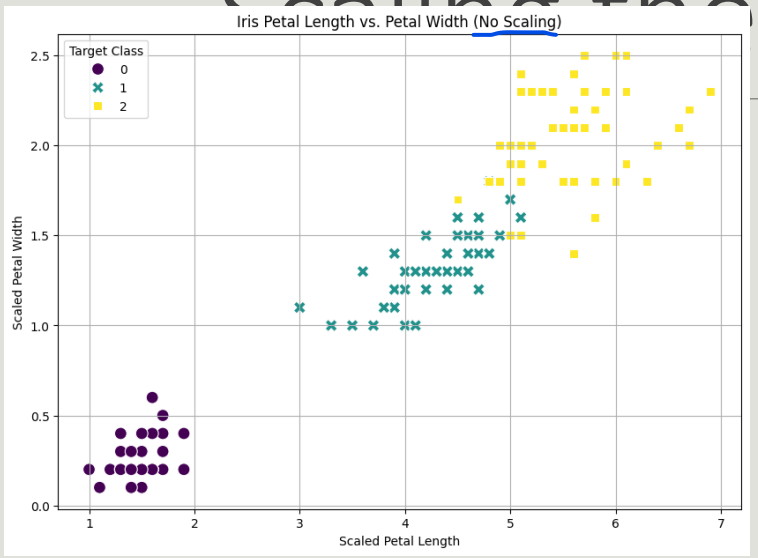
Description
pal
pal
tal
tal
he Iris family

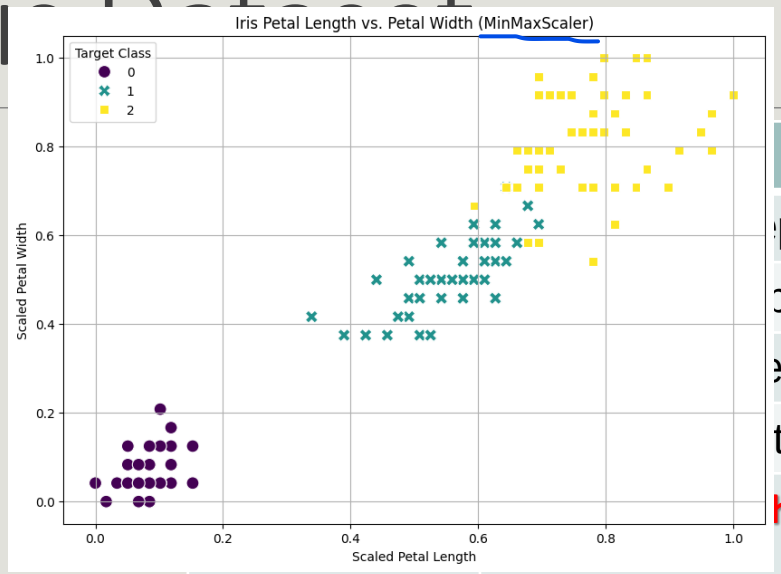


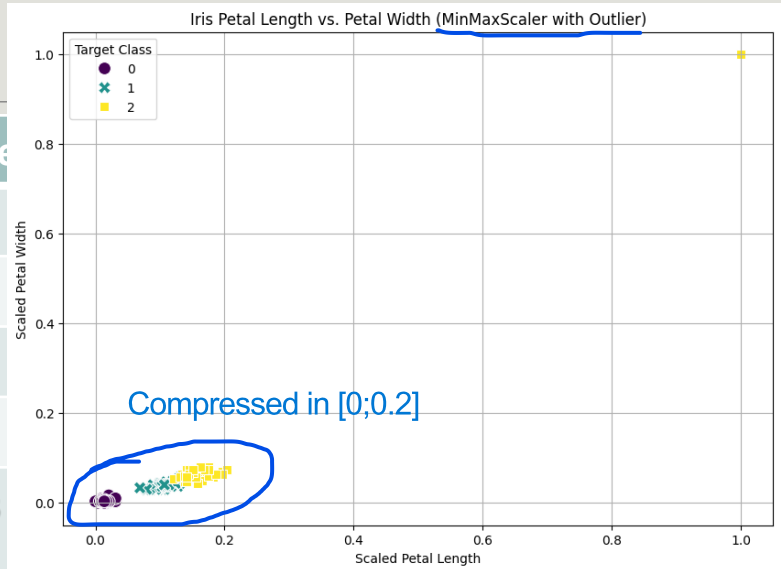


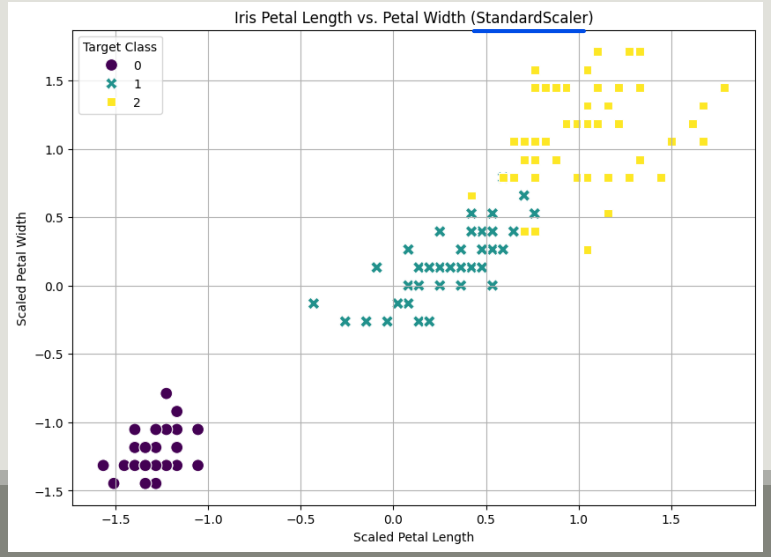
Adding an outlier in [30;30]

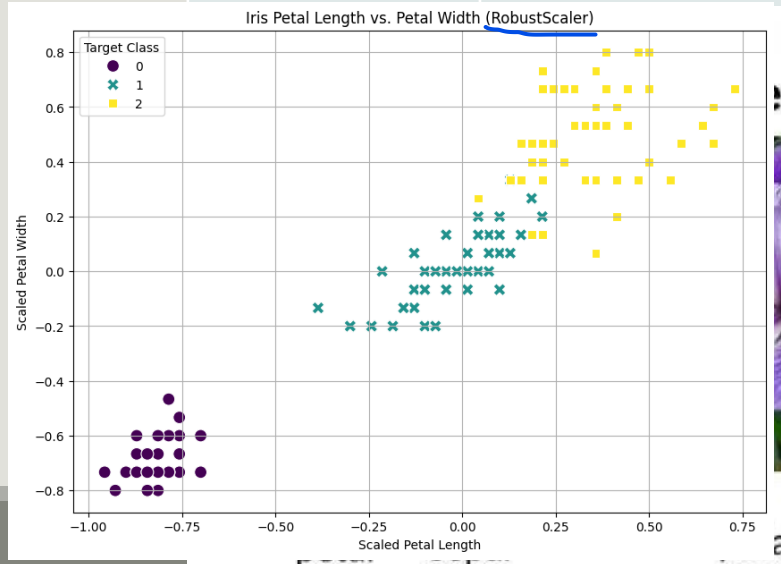
Scaling the Iris Dataset



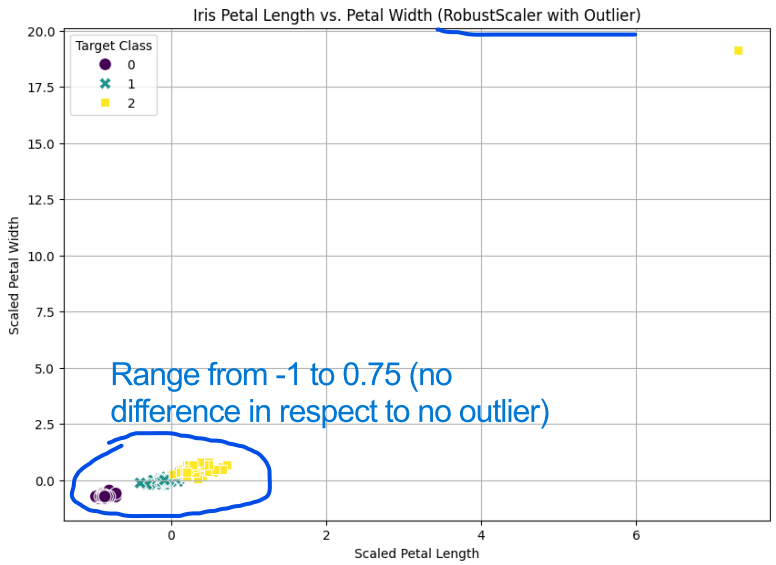












Scaling Bank Data

Example

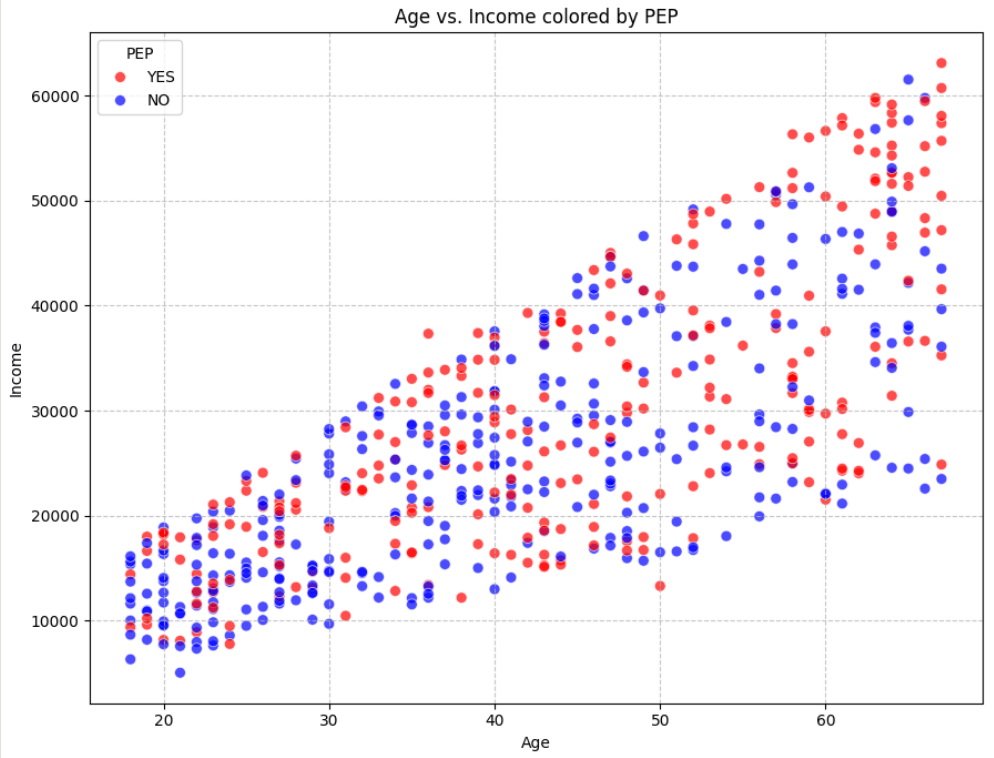
bank-data.csv dataset

- ✓ 600 instances
- ✓ No missing data

Attribute	Description
Id	Unique identifier
Age	Customer age in years (numeric)
Sex	MALE / FEMALE
Region	inner_city/rural/suburban/town
Income	Customer income (numeric)
children	Number of children? (numeric)
car	Does she own a car? (YES/NO)
save_acct	Does she have a save account? (YES/NO)
current_acct	Does she have a current account? (YES/NO)
Mortgage	Does she subscribed a mortgage?(YES/NO)
Pep	**Have she purchased a Personal Equity Plan (PEP) since the last mailing? (YES/NO)**

A Personal Equity Plan (PEP) was a tax-advantaged investment account promoted by the United Kingdom Government to (1) Encourage Household Saving; (2) Promote Long-Term Investment; (3) Support Capital Markets

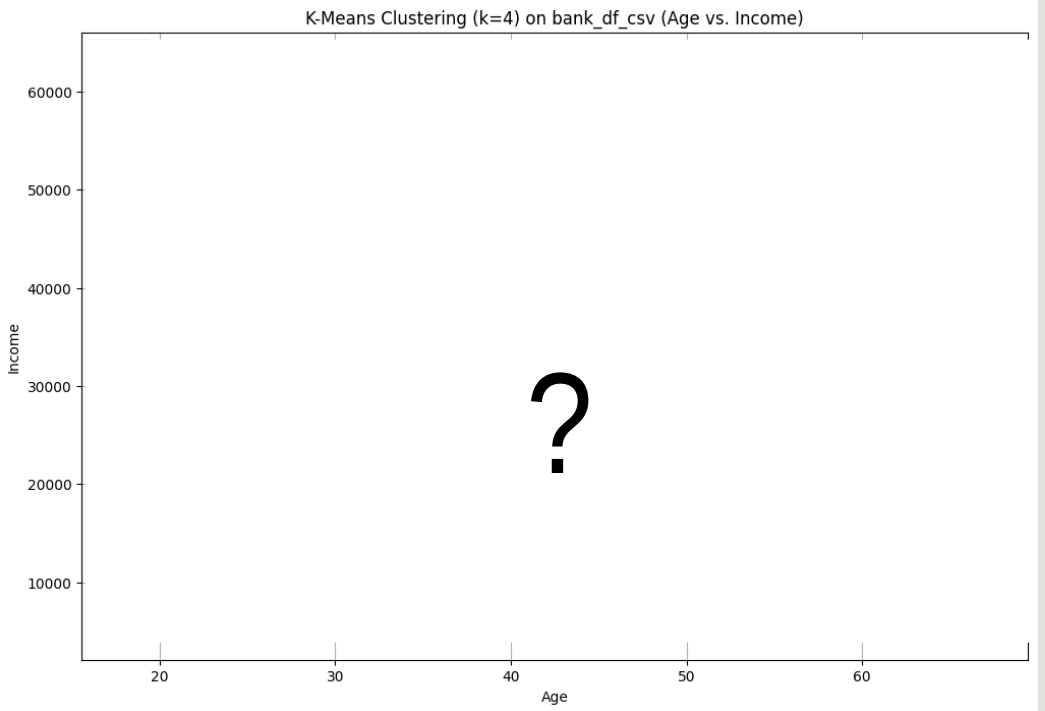
Scaling Bank Data

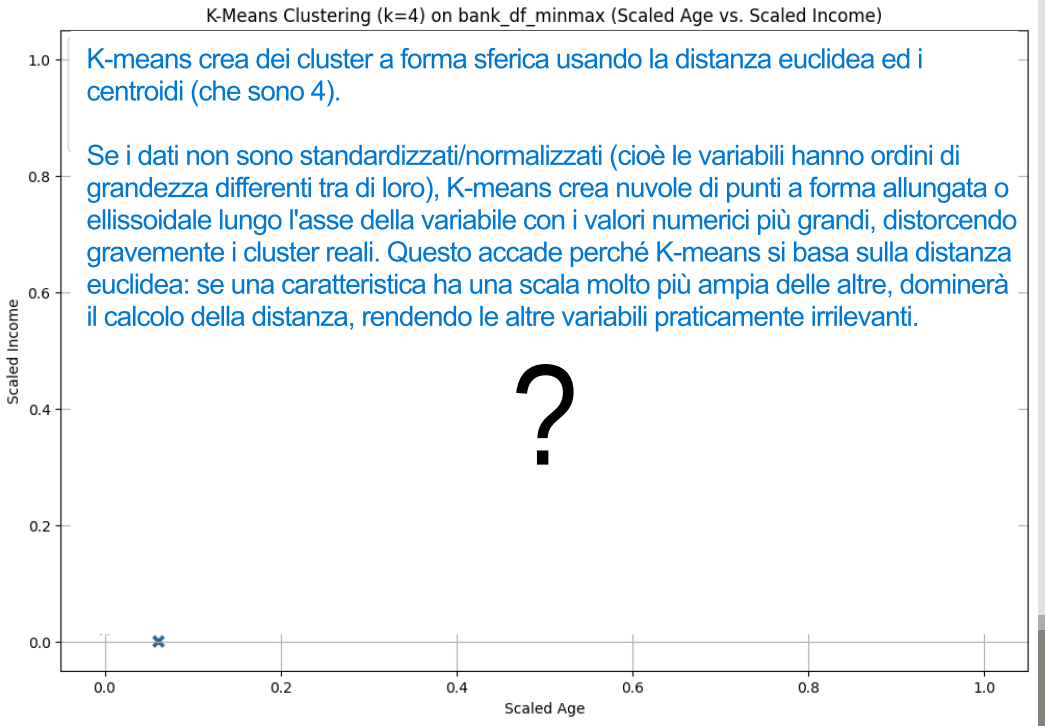


Attribute	
d	ue
ge	m
ex	MALE /
eg	inner_c
ncom	Custom
hildren	be
ar	sh
ave_acct	Does sh
urrent_acct	Does sh
Mortgage	Does sh
Pep	Have sh since th

K-Means k=4

Encourage Household Saving; (2) Promote Long-Term Investment; (3) Support Capital Markets





K-means crea dei cluster a forma sferica usando la distanza euclidea ed i centroidi (che sono 4).

Se i dati non sono standardizzati/normalizzati (cioè le variabili hanno ordini di grandezza differenti tra di loro), K-means crea nuvole di punti a forma allungata o ellissoidale lungo l'asse della variabile con i valori numerici più grandi, distorcendo gravemente i cluster reali. Questo accade perché K-means si basa sulla distanza euclidea: se una caratteristica ha una scala molto più ampia delle altre, dominerà il calcolo della distanza, rendendo le altre variabili praticamente irrilevanti.

Clusters basati prevalentemente sulla differenza di Income (maggior scala rispetto all'età)

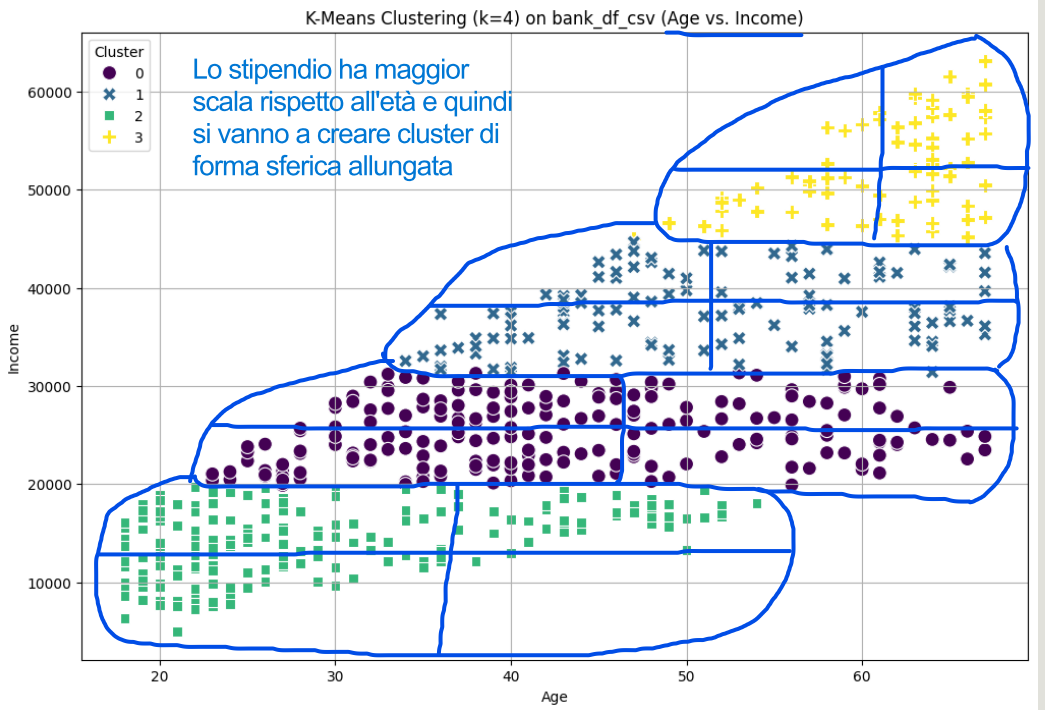
Scaling Bank Data

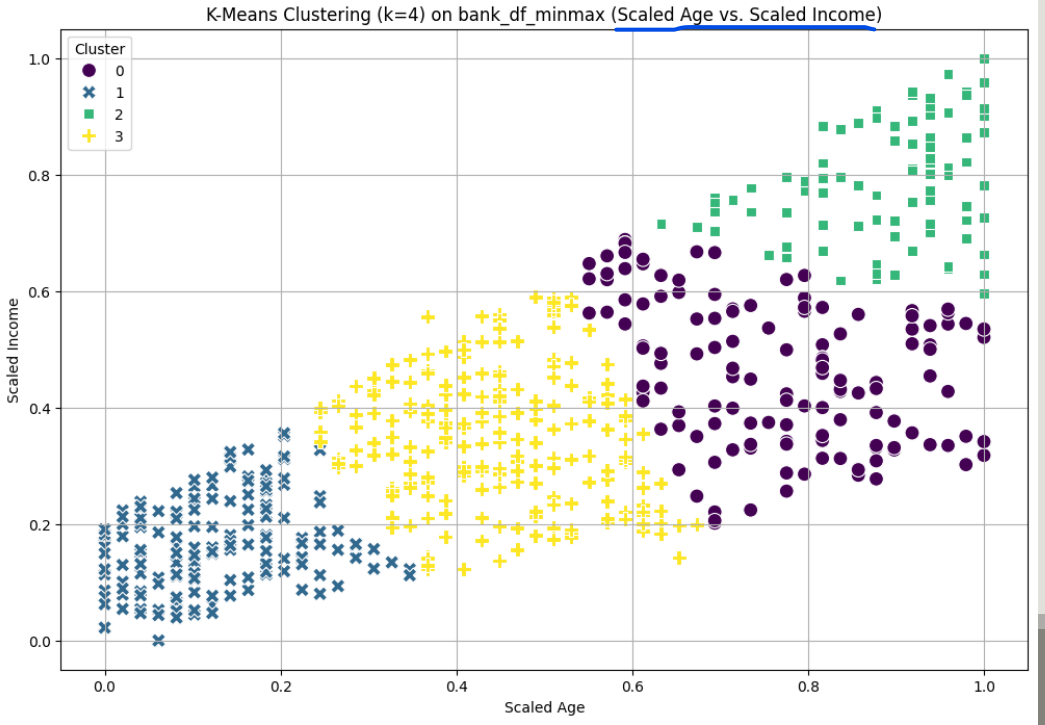


Attribute	
d	ue
ge	m
ex	MALE /
ldren	inner_c
ncom	Custom
ar	be
ave_acct	sh
urrent_acct	Does sh
Mortgage	Does sh
Pep	Have sh since th

Clustering
K-Means
k=4

Encourage Household Saving; (2) Promote Long-Term Investment; (3) Support Capital Markets





Sampling is Everywhere in ML

Sampling appears in:

- Representation of the population
- Train/test split
- Class balancing
- Active learning loops
- LLM pretraining corpora selection
- .....

Sampling works until "*the sample is representative of the real population*".
More formally:

"*the sample behaves like it was drawn from the same probability distribution as the population*"

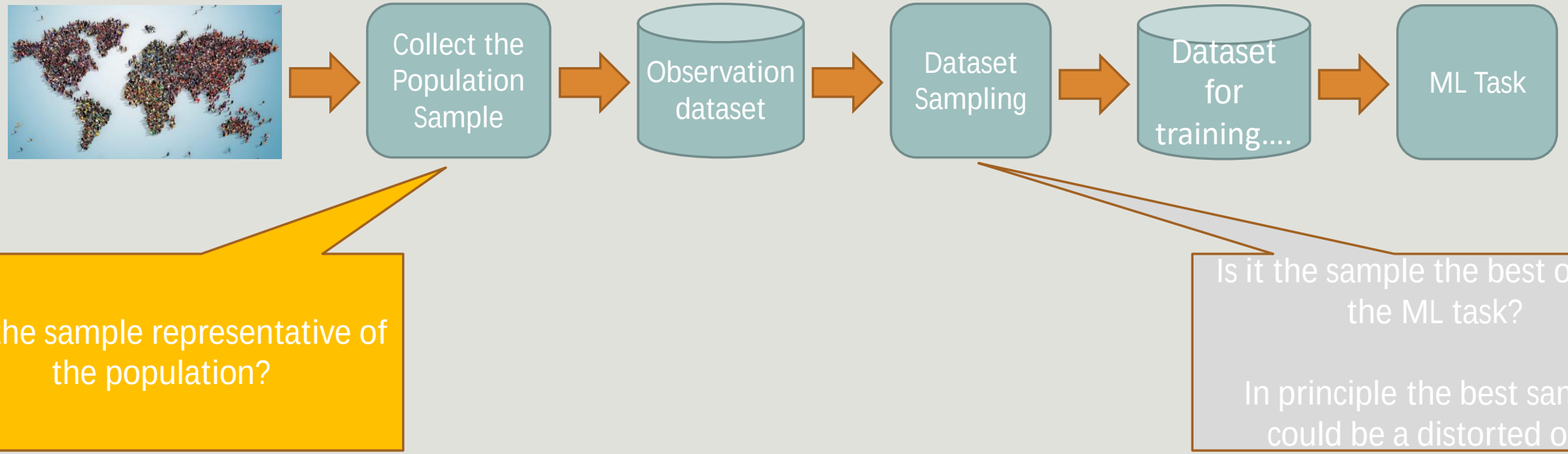
- ✓ Population distribution: $P(X, Y)$
- ✓ Sample distribution: $\hat{P}(X, Y)$

$$P(X, Y) \approx \hat{P}(X, Y)$$

Sampling the Population vs Sampling the Observed Data Set

Let we split the process since they entail different issues

- Population sampling is aimed at creating the initial data set that must be representative of the population
- Dataset Sampling creates the dataset that will be used for the ML task

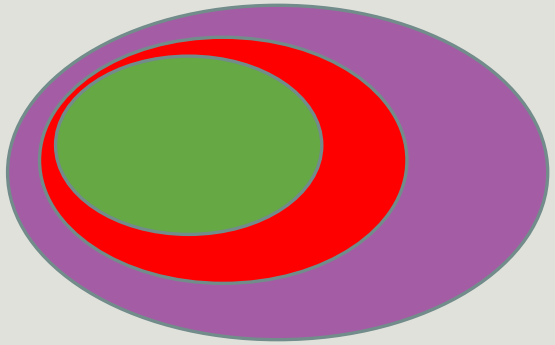


Collecting the Population Sample

A model cannot learn information that is absent from the collected data
A model learn the world described by the sample that can differ from the real population

In order to correctly create the Observation dataset we have to define first

- Population: the entire group of individuals, objects, or events you want to study
- Target Population: the cases on which the model will be deployed
- Sampling Frame: the examples we can actually access



The gap between target population and sampling frame is often the first source of selection bias

Bias Category	Specific Bias	Main cause
Coverage bias	Convenience sampling bias	Data are collected because they are easy or inexpensive to obtain
	Survivorship bias	Only the surviving or successful observations are included
	Coverage bias	The sampling frame does not adequately cover the target population
	Self-selection bias	Individuals decide whether to participate, producing a non-representative sample
Historical bias	Historical bias	The collected data reflect past conditions or societal biases that may no longer hold

Bias Examples

Example 1: Autonomous driving

Target population: Daytime road scenes encountered by autonomous vehicles

Sampling frame: Images collected only in Milan during summer.

The dataset lacks:
- rural roads
- mountain roads
- snow
- heavy rain
- narrow village streets

A pedestrian detector trained on this dataset may perform well in Milan but poorly in the Alps.

Example 2: Prescriptive Watering

Target population: field data from orchards in summer time

Sampling frame: field data collected from one kiwi field in July

The dataset lacks:
- different types of terrain (Clay soils vs. sandy soils)
- different crop behaviors (Peach vs Kiwi)
- different phenological phase of the crop (kiwi in June behaves different than in July, August and September)

A model forecasting humidity reduction trained on this dataset may perform poorly in August in a peach orchard

Bias Examples

Example 1: Autonomous driving

Target population: Daytime road scenes encountered by autonomous vehicles

Sampling frame: Images collected only in Milan during summer.

The dataset lacks:
- rural roads
- mountain roads
- snow
- heavy rain
- narrow village streets

A pedestrian detector trained on this dataset may perform well in Milan but poorly in the Alps.

Coverage bias

Example 2: Prescriptive Watering

Target population: field data from orchards in summer time

Sampling frame: field data collected from one kiwi field in July

The dataset lacks:
- different types of terrain (Clay soils vs. sandy soils)
- different crop behaviors (Peach vs Kiwi)
- different phenological phase of the crop (kiwi in June behaves different than in July, August and September)

A model forecasting humidity reduction trained on this dataset may perform poorly in August in a peach orchard

Coverage bias

Bias Examples

Example 3: Trajectory Analysis

Target population: People traveling around Milan

Sampling frame: GPS trajectories collected free telephone Apps

Types of people (Old/Young, Male/Female) depends on the producing apps (gaming, Meteo, Recipes) that is unknow

An analysis based on such a dataset may over/under represent some categories

Example 4: Analysis of Gym Workout behaviors

Target population: athletes who work out at the gym

Sampling frame: exercises collected through the equipment login system

The dataset lacks in:

- free body exercises that must be entered manually by the athlete
- Undone exercises due to broken or crowded equipments

An analysis based on such a dataset may under represent athletes who prefer free body exercises

Bias Examples

Example 3: Trajectory Analysis

Target population: People traveling around Milan

Sampling frame: GPS trajectories collected free telephone Apps

Types of people (Old/Young, Male/Female) depends on the producing apps (gaming, Meteo, Recipes) that is unknow

An analysis based on such a dataset may over/under represent some categories

Coverage bias

Example 4: Analysis of Gym Workout behaviors

Target population: athletes who work out at the gym

Sampling frame: exercises collected through the equipment login system

The dataset lacks in:

- free body exercises that must be entered manually by the athlete
- Undone exercises due to broken or crowded equipments

An analysis based on such a dataset may under represent athletes who prefer free body exercises

Self-selection bias **Coverage bias**

Bias Examples

Example 5: Gas Consumption Forecast

Target population: All gas contracts in a specific area

Sampling frame: Time-series of 15-minute gas meters from HERA control-units in 2019

- COVID lockdown completely changed people behavior
- The rise in prices caused by the oil crises is changing people's behavior

A model trained in 2019 may poorly perform in a 2020 and in 2022 (war in Ukraine)

Example 6: Restaurant Customer Satisfaction

Target population: All customers

Sampling frame: feedbacks submitted through the website and social

Customers who choose to leave a review are typically those who had either a very positive or a very negative experience

An analysis based on such a dataset may under represent customers with an average experience

Bias Examples

Example 5: Gas Consumption Forecast

Target population: All gas contracts in a specific area

Sampling frame: Time-series of 15-minute gas meters from HERA control-units in 2019

- COVID lockdown completely changed people behavior
- The rise in prices caused by the oil crises is changing people's behavior

A model trained in 2019 may poorly perform in a 2020 and in 2022 (war in Ukraine)

Historical bias

Example 6: Restaurant Customer Satisfaction

Target population: All customers

Sampling frame: feedbacks submitted through the website and social

Customers who choose to leave a review are typically those who had either a very positive or a very negative experience

An analysis based on such a dataset may under represent customers with an average experience

Self-selection bias

Bias Examples

Example 6: Fund Returns Forecast

Target population: Any investment fund

Sampling frame: Ten-year time series of *currently* active funds

- Over the course of those ten years, dozens of funds that lost too much money were closed, merged, or delisted

The model makes the investment appear much safer and more profitable than it actually is. Since the defeats are not considered

Survivorship bias

Coverage Bias Detection

A useful approach to detect a bias is to define the key dimensions along which the target population varies

Dimension	Example categories
Geography	urban, suburban, rural
Time	day, night
Weather	sunny, rain, snow, fog
Device	smartphone, DSLR, CCTV
Demographics	age groups, genders, ethnicities
Environment	indoor, outdoor

Then compare the distributions in the dataset with those expected in the target population. Missing or severely underrepresented combinations indicate potential coverage bias.

Coverage Bias Mitigation

Coverage bias is addressed during **data collection**, not by changing the learning algorithm.

Common strategies include:

- Carefully define the target population before collecting data
- Design a sampling plan that explicitly covers important subpopulations
- Use **stratified sampling** to ensure representation across key factors
- Identify gaps through exploratory analysis and collect additional data to fill them
- Continuously monitor production data for previously unseen conditions and expand the dataset accordingly

Random Sampling

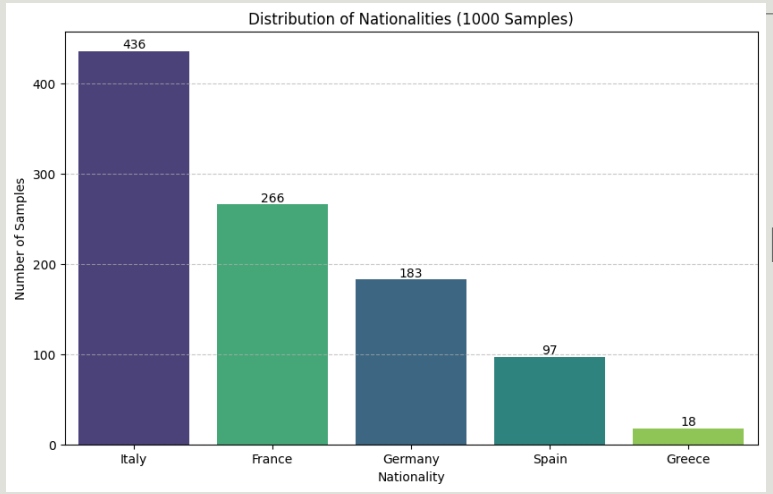
It is the baseline solution: given a population of *N* elements, randomly select a subset of n elements (*n*<<*N*)

- All the data points are equally important
- It does not consider the content or the quality of the data
- Used to reduce the dataset size typically to speed up model training
- It is unbiased from the statistical point of view i.e. $P(X,Y) \approx \hat{P}(X,Y)$ for *n* big enough

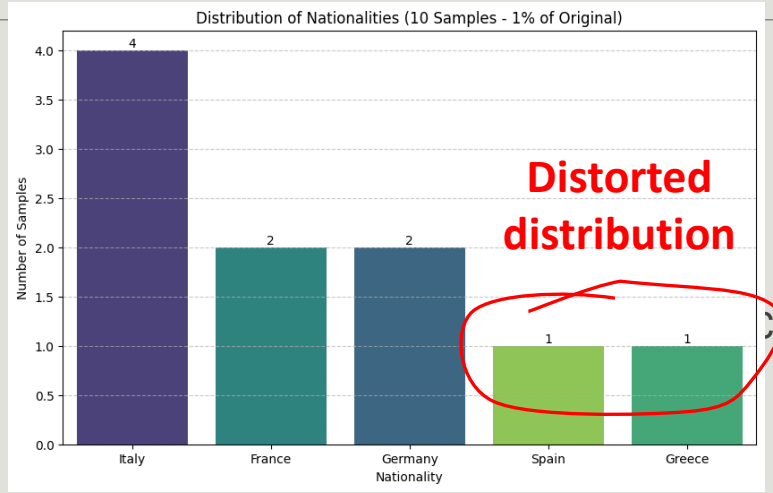
This approach has several drawbacks and limitation if we want to improve the dataset quality

- Is affected by sampling variability
  - i.e. the class proportions are preserved only in expectation, not in every sample
- May Miss Rare Classes
- Ignores Data Content
- Preserves Redundancy

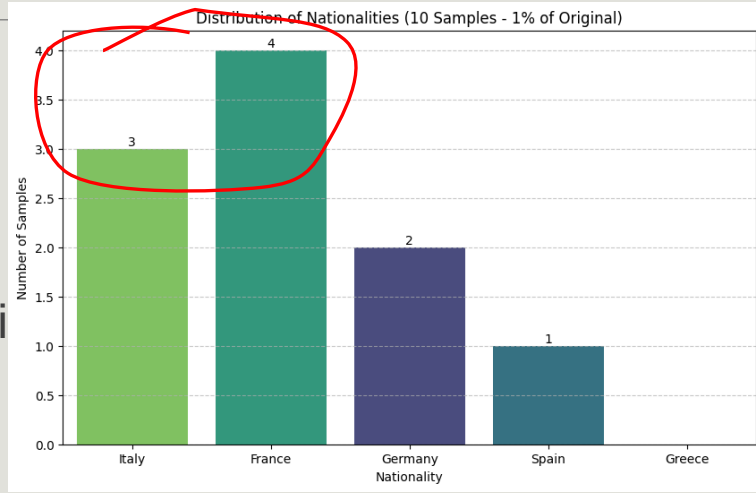
Random Sampling

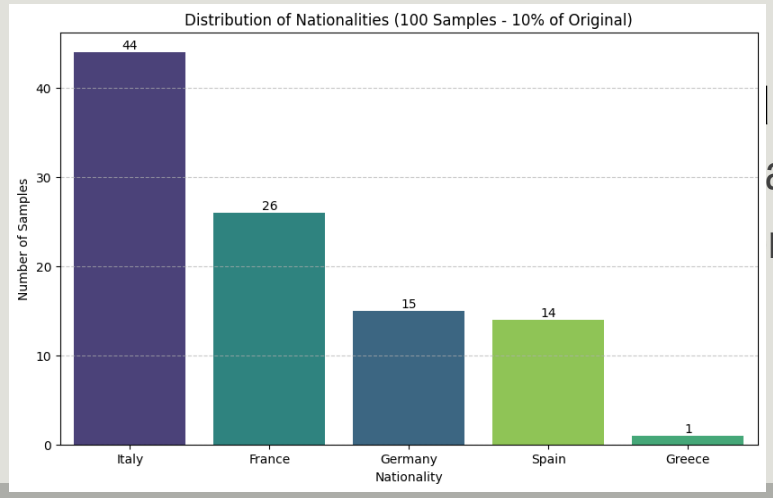


lation



Distorted distribution

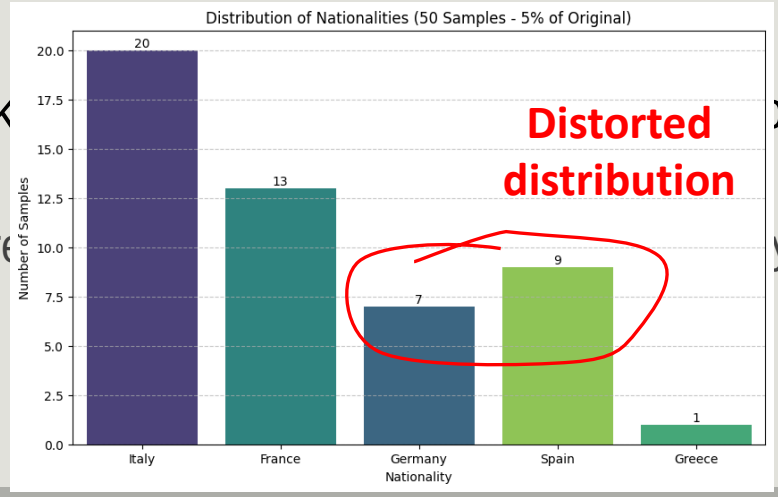




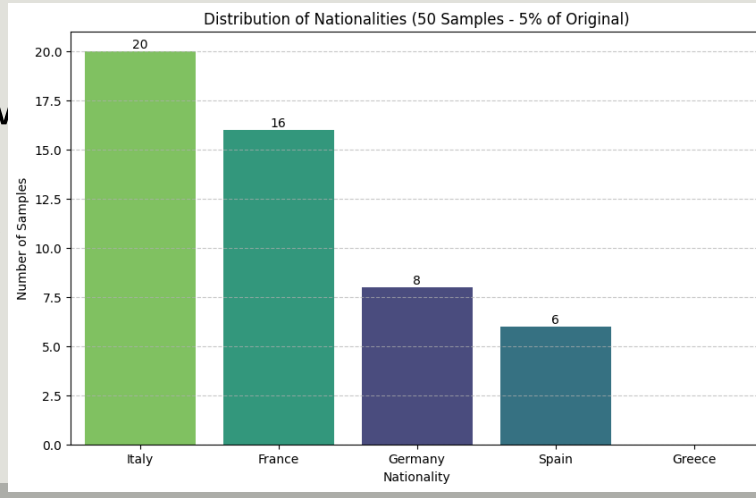
rawback

ability

ns are pre



Run 1

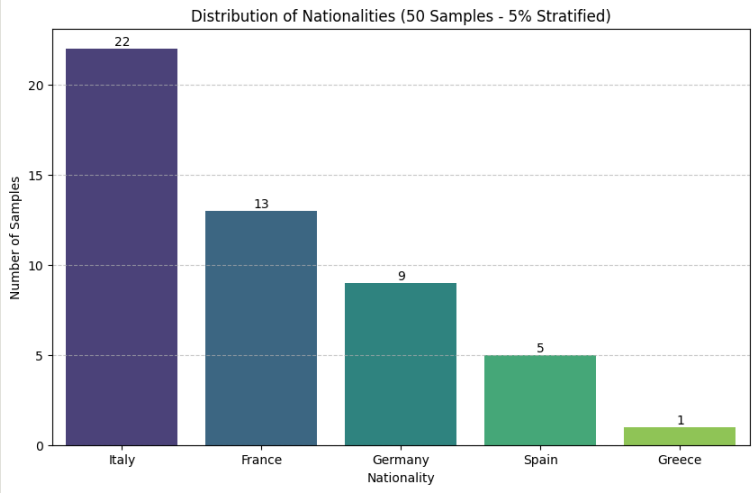


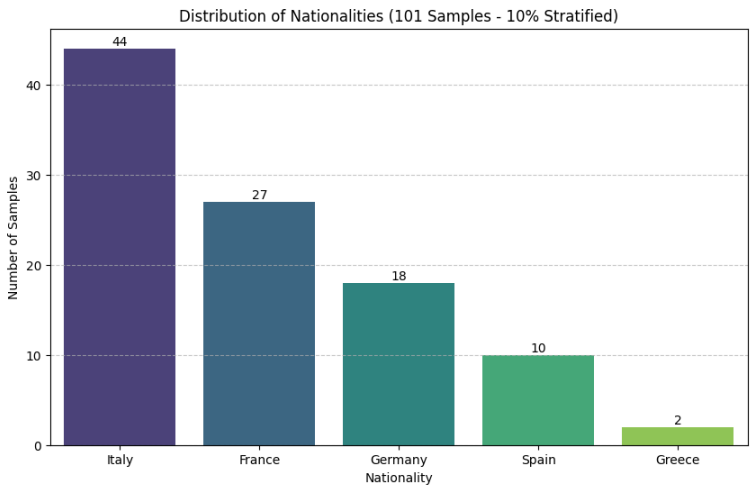
Run 2

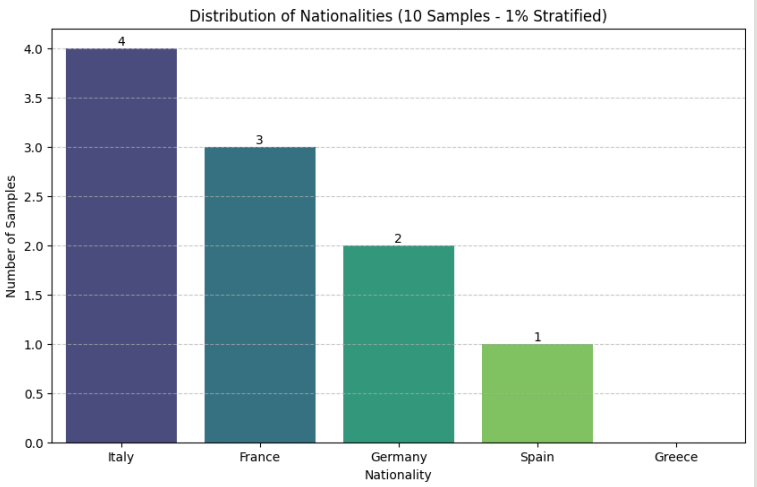
(Proportional) Stratified Sampling

It is particularly useful in presence of rare class of elements
1) The dataset is partitioned in classes through an attribute (e.g. gender, age, category)
2) From each partition a (proportional) percentage of random samples is taken

Stratified sampling provides control over subgroup representation, but guarantees the inclusion of every subgroup only if at least one observation is allocated to each stratum.







Sampling the Population vs Sampling the Observed Data Set

Let we split the process since they entail different issues

- Population sampling is aimed at creating the initial data set that must be representative of the population
- Dataset Sampling creates the dataset that will be used for the ML task



Collect the Population Sample

Observation dataset

Dataset Sampling

Dataset for training....

ML Task

Is it the sample representative of the population?

Is it the sample the best one for the ML task?

In principle the best sample could be a distorted one

Sampling depends on the AI task

In classical statistics, the goal of sampling is to faithfully represent the population. In Data-Centric AI, the goal of sampling is to create a dataset that is maximally informative for the learning task.

Consequently, the optimal training dataset is often intentionally different from the population distribution, whereas the evaluation dataset should closely resemble the deployment distribution.

Dataset purpose	Sampling strategy
Estimate population statistics	Representative random sample of the whole population
Detect rare events	Over sample rare cases
Improve robustness	Over represent difficult examples
Fairness analysis	Balance demographic groups
Estimate deployment performance	Representative test set of the whole population

Oversampling Techniques

Random Oversampling: randomly duplicate examples from the minority class until the desired class distribution is achieved.

Advantages
- Simple

Drawbacks
- no new information is introduced
- increases the risk of overfitting
- duplicated examples reduce diversity

Synthetic Minority Over-sampling Technique (SMOTE): rather than duplicating observations creates new minority examples by interpolating between existing ones.

Advantages
- Reduces overfitting
- Enriches the minority region
- Widely used in tabular data

Drawbacks
- Synthetic examples may be unrealistic
- May blur class boundaries
- Less effective for images or text

Oversampling Techniques

Random Oversampling: randomly duplicate examples from the minority class until the desired class

Random oversampling increases the risk of overfitting because it increases the weight of existing examples without necessarily ***adding new information***.
Minority-class examples are duplicated. The model repeatedly sees the same observations and may fit their specific characteristics too closely, including noise and outliers, rather than learning patterns that *generalize to unseen data*.

Drawbacks

- no new information is introduced
- increases the risk of overfitting
- duplicated examples reduce diversity

que (SMOTE): rather than duplicating observations creates new
een existing ones.

- Reduces overfitting
- Enriches the minority region
- Widely used in tabular data

Drawbacks

- Synthetic examples may be unrealistic
- May blur class boundaries
- Less effective for images or text

Oversampling Techniques

Active Learning: it is the training model that guide data collection and prioritize regions where it frequently makes mistakes. The dataset grows exactly where the model needs more information.

- Uncertainty Sampling – the model asks for help when it is unsure
- Diversity Sampling – the dataset should cover the feature space
- Failure-Driven Data Collection – collect more data where the deployed model actually fails.

Data Augmentation: for structured signals such as images, audio, or text, new examples are generated by applying transformations that preserve the label (e.g. for images rotation, translation, flipping)
Unlike SMOTE, augmentation exploits domain knowledge to produce plausible variations.

Synthetic Minority Over-sampling Technique

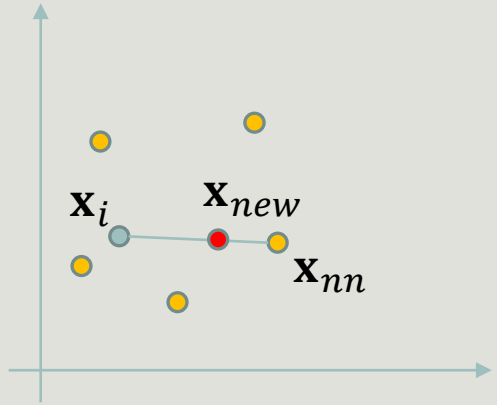
SMOTE generates **new synthetic samples** by interpolating between neighboring examples of the minority class. The objective is to increase the representation of the minority class while reducing the risk of overfitting associated with duplicated observations.

Let $D_m = \{\mathbf{x}_1, \ldots \mathbf{x}_n\}$ be the set of minority-class examples.
For each minority example $\mathbf{x}_i$

Step 1: Compute the *k* nearest neighbors of $\mathbf{x}_i$ among the minority-class samples only.
Step 2: Randomly choose one neighbor $\mathbf{x}_{nn}$
Step 3: Generate a synthetic sample

$$\mathbf{x}_{new} = \mathbf{x}_i + \lambda\,(\mathbf{x}_{nn} - \mathbf{x}_i)$$

where λ is a random number uniformly distributed between 0 and 1.



SMOTE works with numerical or ordinal data. If nominal data are involved the most common variation is SMOTE-NC, where categorical features are assigned using the most common values between the neighbor

Measuring Performances with Oversampling

Training data may be deliberately distorted to improve learning, but the test data should remain representative of the deployment population. This brings two side effects:

- Testing a model on a data distribution that differs from the one of the training data will lead to lower overall accuracy
- Since the target population is highly imbalanced, overall accuracy is often misleading.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Consider a binary classification problem in which

- # Record of class 0 = 9990
- # Record of class 1 = 10

A model that always returns class 0 will have an accuracy of 9990/10000 = 99.9%

Measuring Performances with Oversampling

A better approach is to compute **class-specific metrics**.

Precision measures the fraction of record results actually positive among all those who were classified as such

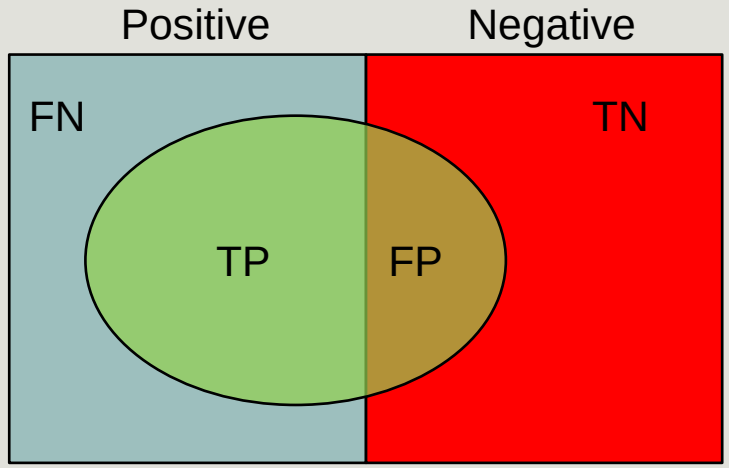
- High values indicate that few negative class records were incorrectly classified as positive.

Recall measures the fraction of positive records correctly classified

- High values indicate that few records of the positive class were incorrectly classified as negatives.

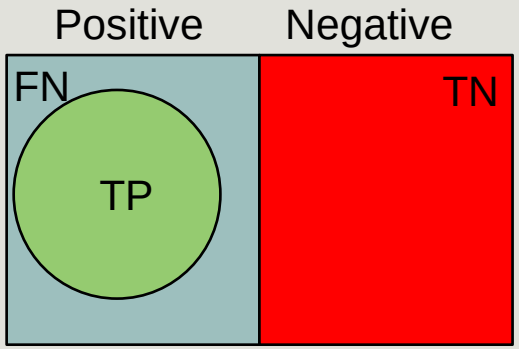
$$\text{Precision, p} = \frac{TP}{TP + FP}$$

$$\text{Recall, r} = \frac{TP}{TP + FN}$$

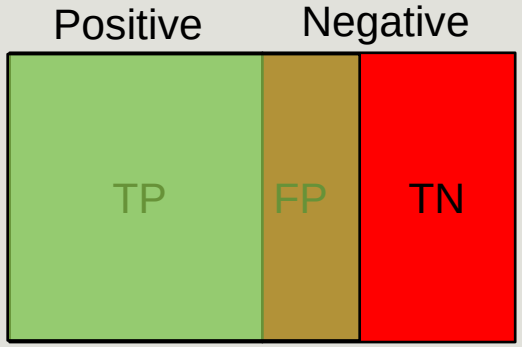


Precision and Recall

Precision = 1 there are no false positives



Recall = 1 if there are no false negatives



If both are valid 1, the predetermined classes coincide with the real ones

F-measure

A metric that summarizes precision and recall is called F-measure

$$\text{F-measure, F} = \frac{2rp}{r+p} = \frac{2\times TP}{2\times TP + FP + FN}$$

F-measure represents the harmonic mean of precision and recall

- The harmonic average between two x and y numbers tends to be close to the smallest of the two numbers. So if the harmonic average is high, it means both precision and recall are.
- If F=1 there have been no false negative or false positives

Impact of oversampling on measures

When oversampling you should generally expect the model to become **better at detecting the minority class (TP)**, but this often comes with a trade-off: it may generate more false positives (FP).

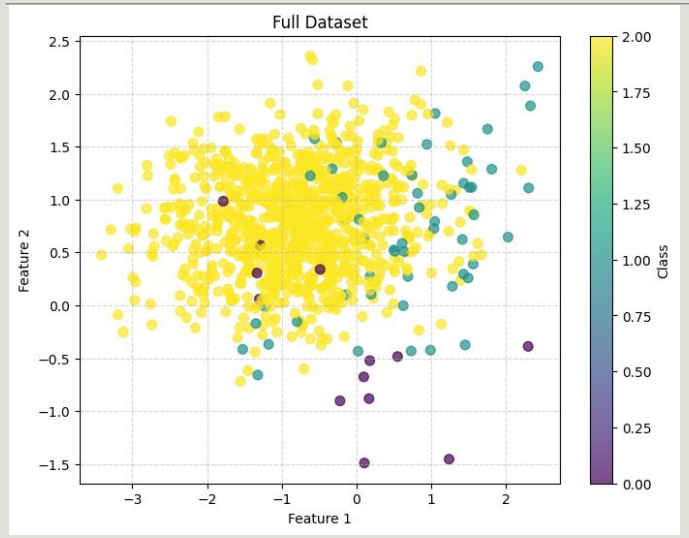
$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

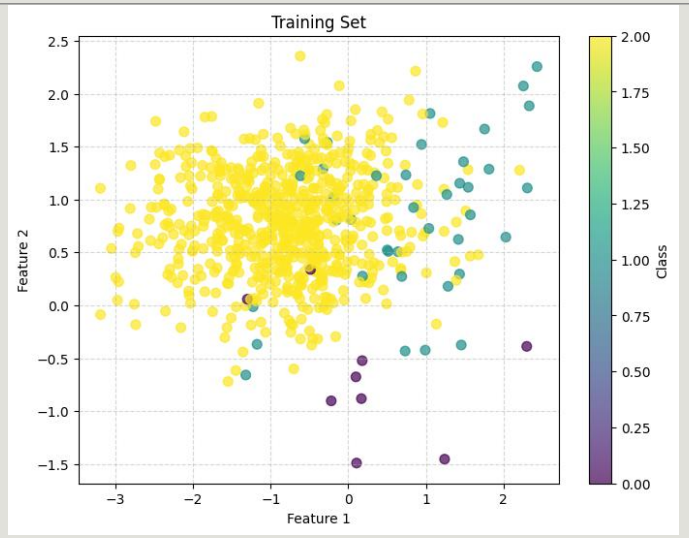
$$\text{Precision, p} = \frac{TP}{TP + FP}$$

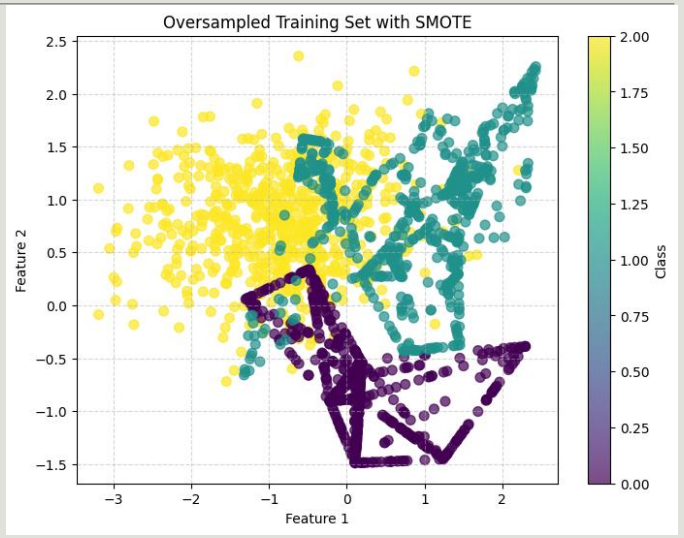
$$\text{Recall, r} = \frac{TP}{TP + FN}$$

Metric	Expected after Oversampling	Why
Accuracy	↔ or ↓	More minority cases are predicted, potentially increasing false positives
Precision	↔ or ↓	Some additional positive predictions may be incorrect
Recall	↑	The classifier becomes better at finding minority examples
F1-score	Usually ↑	Improved recall can outweigh the loss in precision

SMOTE Examples



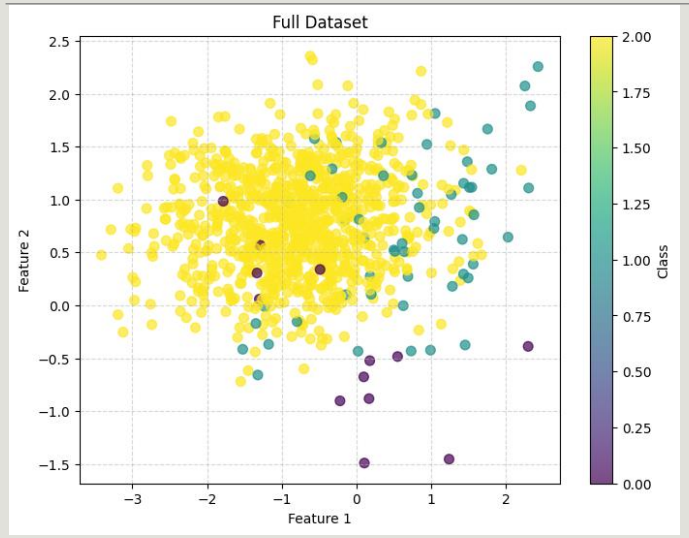




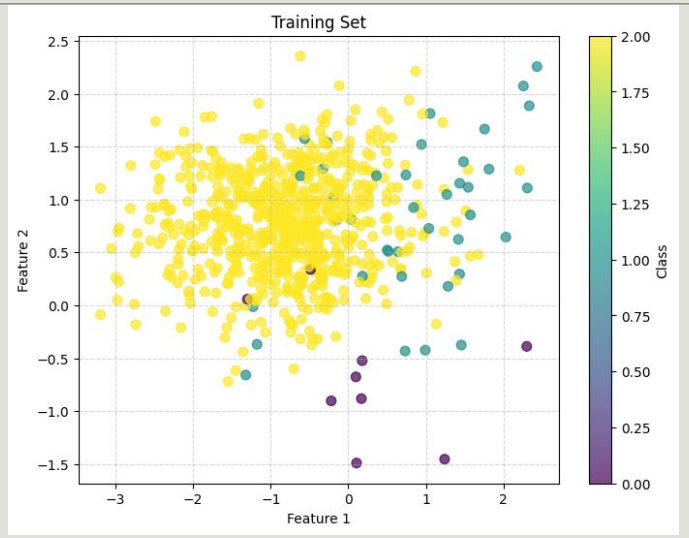
Class 0: 1.3% of examples
Class 1: 5.5% of examples
Class 2: 93.2% of examples

Class 0 652 examples
Class 1 652 examples
Class 2 652 examples

SMOTE Examples



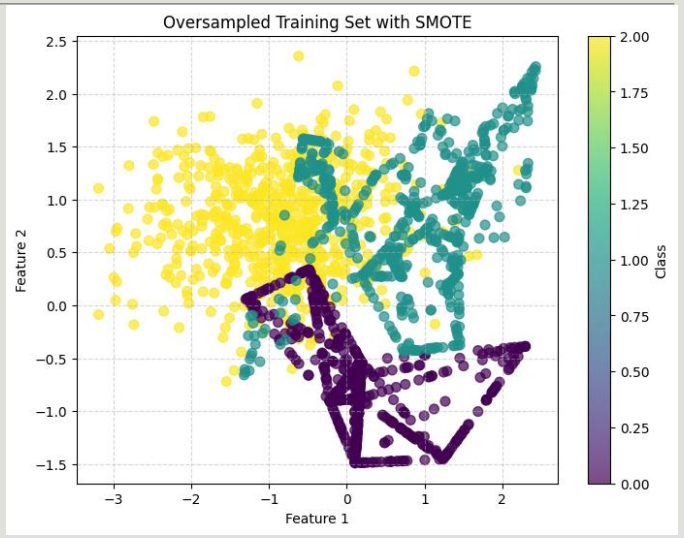
Class 0: 1.3% of examples
Class 1: 5.5% of examples
Class 2: 93.2% of examples



Accuracy: 0.9000

Classification Report:

	precision	recall	f1-score	support
Class 0	**0.00**	**0.00**	0.00	4
Class 1	**0.14**	**0.12**	0.13	16
Class 2	**0.94**	0.96	0.95	280

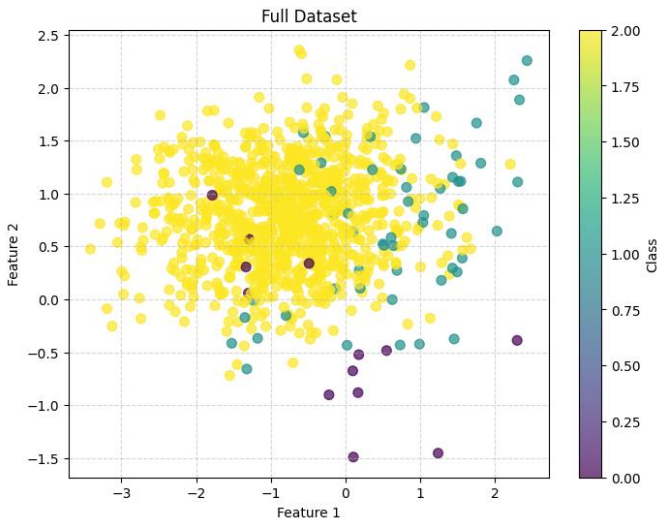


Accuracy (with SMOTE): **0.7567**

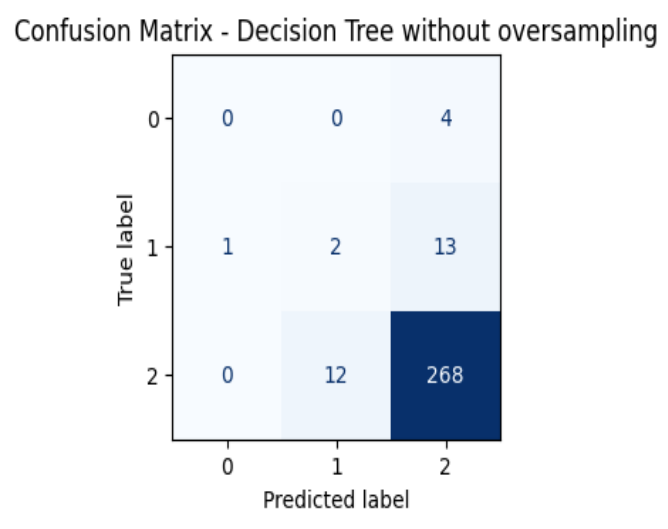
Classification Report (with SMOTE):

	precision	recall	f1-score	support
Class 0	0.06	**0.25**	0.10	4
Class 1	0.16	**0.56**	0.25	16
Class 2	0.96	**0.78**	**0.86**	280

SMOTE Examples



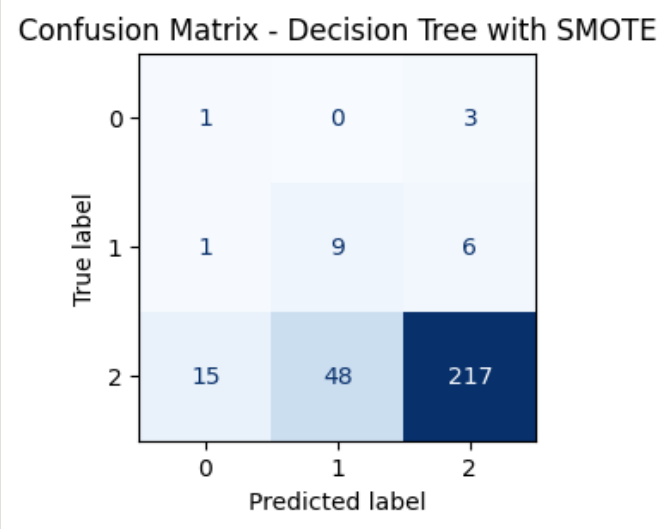
Class 0: 1.3% of examples
Class 1: 5.5% of examples
Class 2: 93.2% of examples



Accuracy: 0.9000

Classification Report:

	precision	recall	f1-score	support
Class 0	0.00	0.00	0.00	4
Class 1	0.14	0.12	0.13	16
Class 2	0.94	0.96	0.95	280



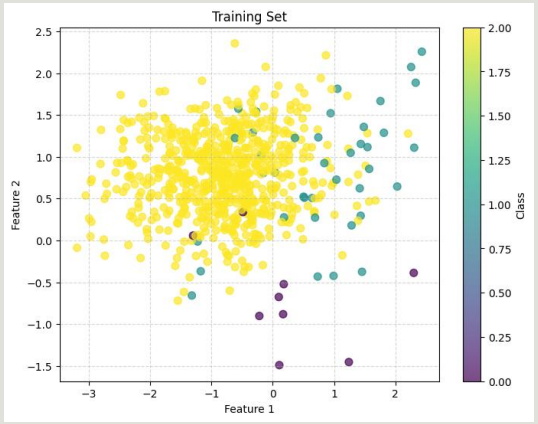
Accuracy (with SMOTE): **0.7567**

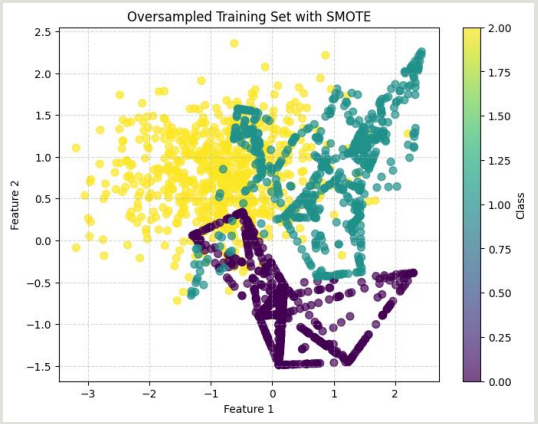
Classification Report (with SMOTE):

	precision	recall	f1-score	support
Class 0	0.06	0.25	0.10	4
Class 1	0.16	0.56	0.25	16
Class 2	0.96	0.78	0.86	280

SMOTE Limitations: class overlaps

SMOTE tends to create linear patterns since it repeatedly linearly interpolates dataset points





SMOTE does not distinguish the position of minority points. In case of class overlap synthetic minority points may be generated inside majority-class regions.

Borderline SMOTE analyzes minority points neighborhood and distinguishes between:

- Mostly minority (Safe): does not require oversampling
- Almost majority (Noise): not oversampled
- **Mix of minority + majority(Danger/borderline): oversample here!**

Borderline-SMOTE concentrates on minority observations close to the decision boundary.

Borderline SMOTE Variants

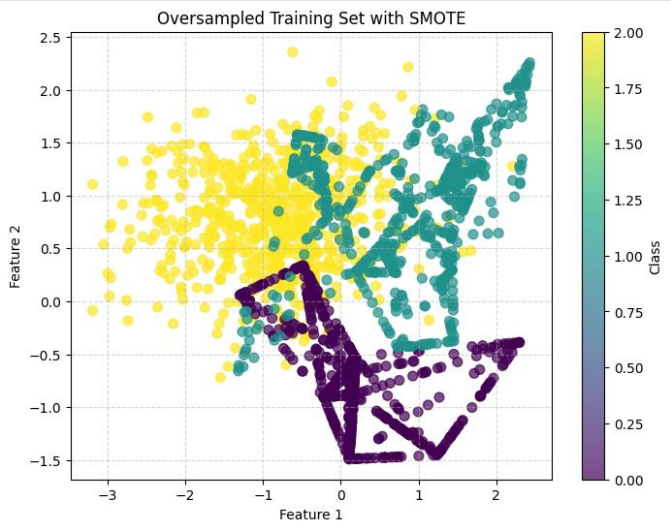
We distinguish two different Borderline SMOTE versions:

- Borderline SMOTE 1: interpolation takes places only between 2 minority points
- Borderline SMOTE 2: it also interpolates toward the nearest majority points, keeping the new point closest to the starting minority point

Thus:

- Borderline SMOTE 1 strengthens the minority class by interpolating among its members;
- Borderline SMOTE 2 extends it to include the majority class as well. However, staying close to the minority point does not guarantee that the new example will be in the correct region: the two methods must be compared on your own dataset.

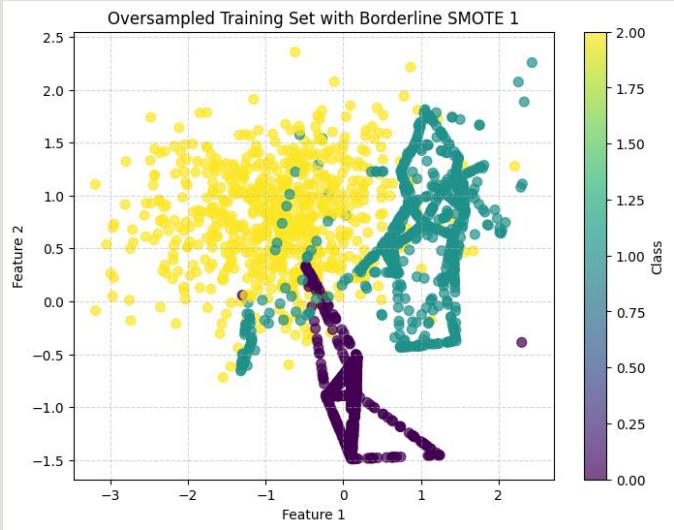
SMOTE Borderline Examples



Accuracy: 0.7567

Classification Report:

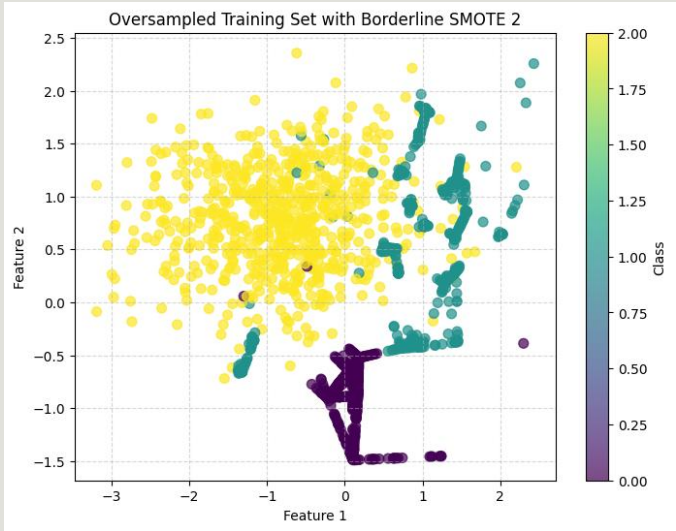
	precision	recall	f1-score	support
Class 0	0.06	0.25	0.10	4
Class 1	0.16	0.56	0.25	16
Class 2	0.96	0.78	0.86	280



Accuracy: 0.8233

Classification Report:

	precision	recall	f1-score	support
Class 0	0.20	0.25	0.22	4
Class 1	0.16	0.44	0.23	16
Class 2	0.96	0.85	0.90	280



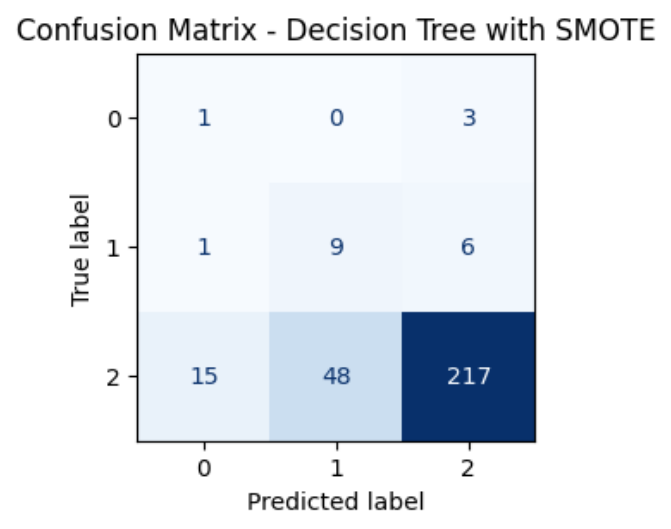
Accuracy: 0.9000

Classification Report:

	precision	recall	f1-score	support
Class 0	0.50	0.25	0.33	4
Class 1	0.28	0.44	0.34	16
Class 2	0.96	0.94	0.95	280

Takeaway: Where you generate matters more than how much. Borderline-SMOTE 2 oversamples only near the decision boundary. It keeps accuracy at 0.90 and gives the best F1 on the rare classes (0.33 and 0.34, against 0.10 and 0.25 with plain SMOTE).

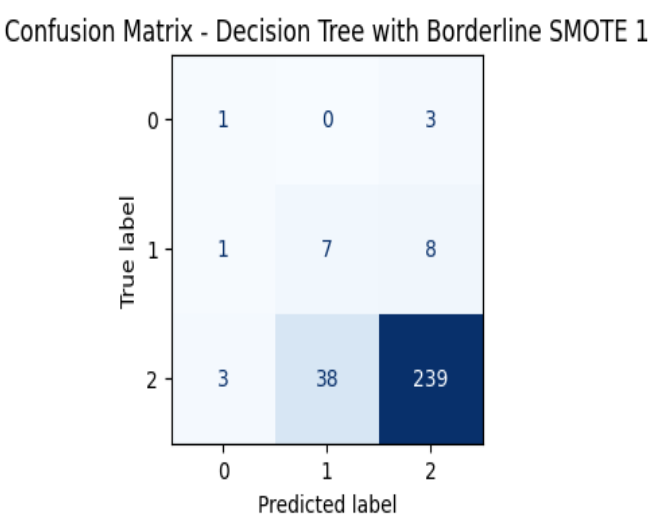
SMOTE Borderline Examples



Accuracy: 0.7567

Classification Report:

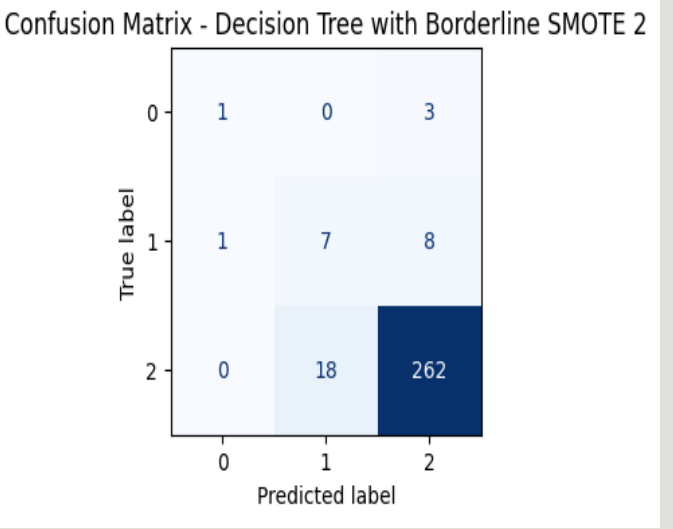
	precision	recall	f1-score	support
Class 0	0.06	0.25	0.10	4
Class 1	0.16	0.56	0.25	16
Class 2	0.96	0.78	0.86	280



Accuracy: 0.8233

Classification Report:

	precision	recall	f1-score	support
Class 0	0.20	0.25	0.22	4
Class 1	0.16	0.44	0.23	16
Class 2	0.96	0.85	0.90	280



Accuracy: 0.9000

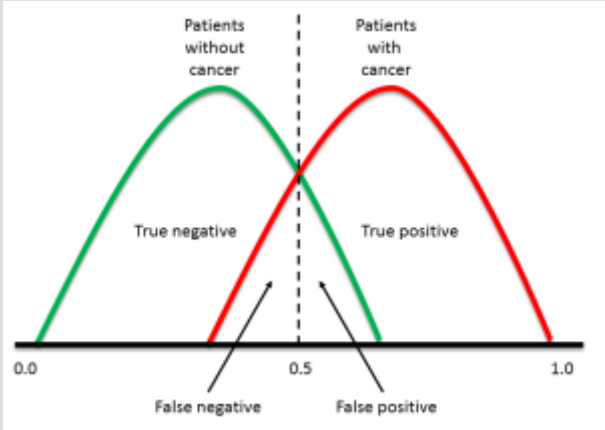
Classification Report:

	precision	recall	f1-score	support
Class 0	0.50	0.25	0.33	4
Class 1	0.28	0.44	0.34	16
Class 2	0.96	0.94	0.95	280

Cost-Based Evaluation

Accuracy, Precision-Recall and F-measure classify an instance as positive if P(+,i)>P(-,i).

- They assume that FN and FP have the same weight, thus they are Cost-Insensitive
- In many domains this is not true!
  - For a cancer screening test, for example, we may be prepared to put up with a relatively high false positive rate in order to get a high true positive, it is most important to identify possible cancer sufferers
  - For a follow-up test after treatment, however, a different threshold might be more desirable, since we want to minimize false negatives, we don't want to tell a patient they're clear if this is not actually the case.



The Cost Matrix

The cost matrix encodes the penalty that a classifier incurs in classifying a record in a different class
A negative penalty indicates the "prize" that is obtained for a correct classification

$$C(M)=TP\times C(+|+) + FN\times C(+|-) + FP \times C(-|+) + TN\times C(-|-)$$

		Predicted Class j	
	C(i|j)	**Class = +**	**Class=-**
Actual Class i	**Class=+**	C(+|+)	C(+|-)
	Class=-	C(-|+)	C(-|-)

A model constructed by structuring, as a purity function, a cost matrix will tend to provide a model with a minimum cost over the specified weights

Computing the Cost

Cost Matrix	PREDICTED CLASS		
ACTUAL CLASS	C(i|j)	+	-
	+	-1	100
	-	1	0

Accuracy = 80%

Cost = 3910

Model M_1	PREDICTED CLASS		
ACTUAL CLASS		+	-
	+	150	40
	-	60	250

Model M_2	PREDICTED CLASS		
ACTUAL CLASS		+	-
	+	250	45
	-	5	200

Accuracy = 90%

Cost = 4255

M1 is preferable despite its lower accuracy

Oversampling vs Cost-sensitive **Learning**

In Cost-sensitive Learning costs are not only used during evaluation, but are made explicit in the loss-function during the model training.

While they pursue the same goal: *increase the system performance on the rare class*; the two approaches follow different philosophies: **Data-Centric** VS **Model-Centric**

- Oversampling: provides the ML-model with new information about the minority class and solves the problem of low coverage of the training set
  - *Errors happen because the model is unable to learn from the examples*
- Cost-Sensitive Learning: does not increase the information available to the ML-model, but makes it explicit during the training phase the importance of an error on the rare class
  - *Errors happen because the model does not understand the relevance of an error (i.e. the loss function does not properly model it)*

If you have enough representative data, using the original (unbiased) dataset together with a cost-sensitive loss is theoretically preferable.

Dataset Shift

A model is trained on data drawn from the Sampling Frame and deployed on the Target Population.

Dataset shift occurs when these two distributions differ, in one of three distinct ways:

$$P_{train}(X, Y) \neq P_{prod}(X, Y) \qquad P(X, Y) = P(Y \mid X)\, P(X) = P(X \mid Y)\, P(Y)$$

Depending on which term of the joint distribution changes, we distinguish:

Type	What differs between Sampling Frame and Target Population	What stays the same
Covariate shift	Distribution of the input features P(X)	Feature → label relationship P(Y|X)
Label shift	Class frequencies P(Y)	Feature patterns per class P(X|Y)
Concept drift	The feature → label relationship P(Y|X)	— (the concept itself has changed)

One-line intuition

Covariate shift: same rules, different world | **Label shift:** same world, different frequencies | **Concept drift:** the rules of the world have changed

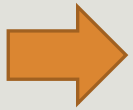
Covariate Shift

The distribution of the input features P(X) in the Target Population differs from the Sampling Frame, but the feature → label relationship P(Y|X) remains the same

- The model is applied to a different region of the feature space than the one it was trained on

Sampling Frame (training)

Autonomous driving images collected in Milan in summer:
sunny weather, urban streets, high image quality



Target Population (deployment)

All road scenes in Italy:
mountain roads, winter fog, rural village streets, snow and rain

Key property: P(Y|X) is unchanged: the rules are the same, but the inputs look different. Re-weighting the training examples to match the Target Population can correct it without relabeling

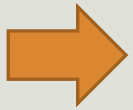
Label Shift (Prior Shift)

The frequency of each class P(Y) in the Target Population differs from the Sampling Frame, but the feature patterns associated with each class P(X|Y) remain the same

- The prior assumptions of the model about class proportions are wrong, not its understanding of each class

Sampling Frame (training)

Fraud detection, 2022: fraud rate 0.5%, normal transactions 99.5%
The model is calibrated to predict fraud very rarely



Target Population (deployment)

During a new fraud campaign: fraud rate 5% (10× higher), normal transactions 95%
What fraud looks like is unchanged, but it occurs far more often

Key property: P(X|Y) is unchanged: the model knows what fraud looks like, but its threshold is calibrated for the wrong base rate. Adjusting the decision threshold or the class priors can fix it without full retraining

Concept Drift

The feature → label relationship $P(Y|X)$ changes in the Target Population: the very concept the model learned no longer holds

- Even identical inputs now produce different outputs because the rules of the world have changed: the most severe form of shift

Example: credit risk scoring

- Sampling Frame: loan data 2022 (stable economy)
  - *"stable job + mid income → repays the loan"*
- Target Population: loans in 2024 (high inflation, rising costs)
  - *"stable job + mid income → default risk has increased"*

The input features are the same, but the rule has changed

Detecting drift: monitor model performance

Month	Accuracy	Status
Jan (baseline)	91%	✓ Normal
Feb	89%	✓ Normal
Mar	85%	⚠ Watch
Apr	79%	✕ **Drift**
May	73%	✕ **Retrain**

Workflow: deploy the model → log predictions → receive ground-truth labels → compute metrics on a rolling window → alert when performance drops below a threshold → retrain

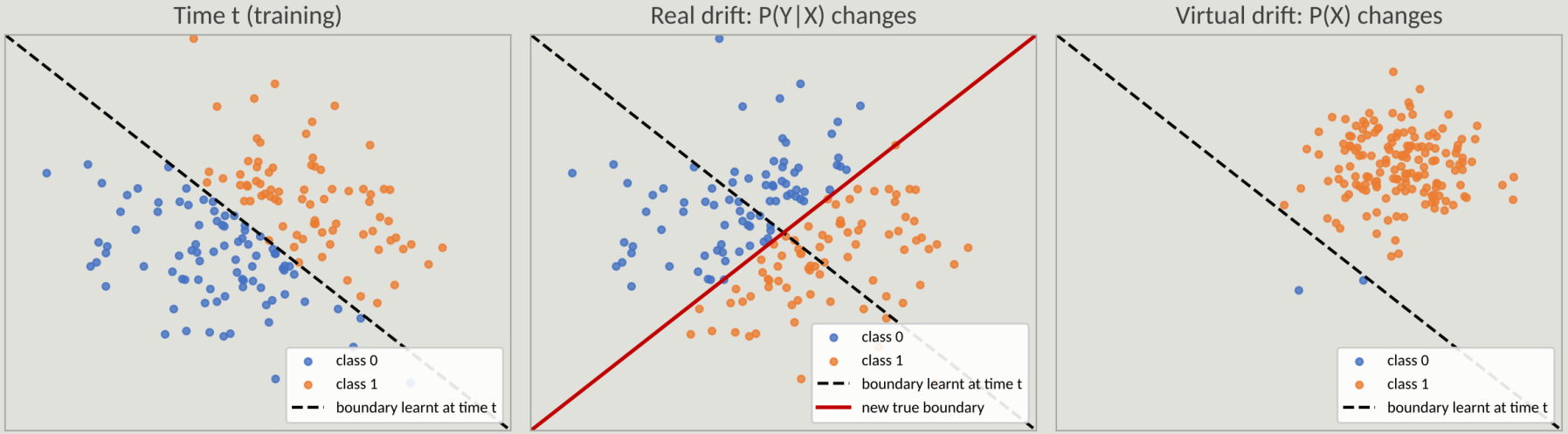
Key challenge: ground-truth labels may arrive with a delay (a loan default takes months to confirm). For an earlier warning, monitor the distribution of the input features with statistical tests such as the Population Stability Index

Concept Drift

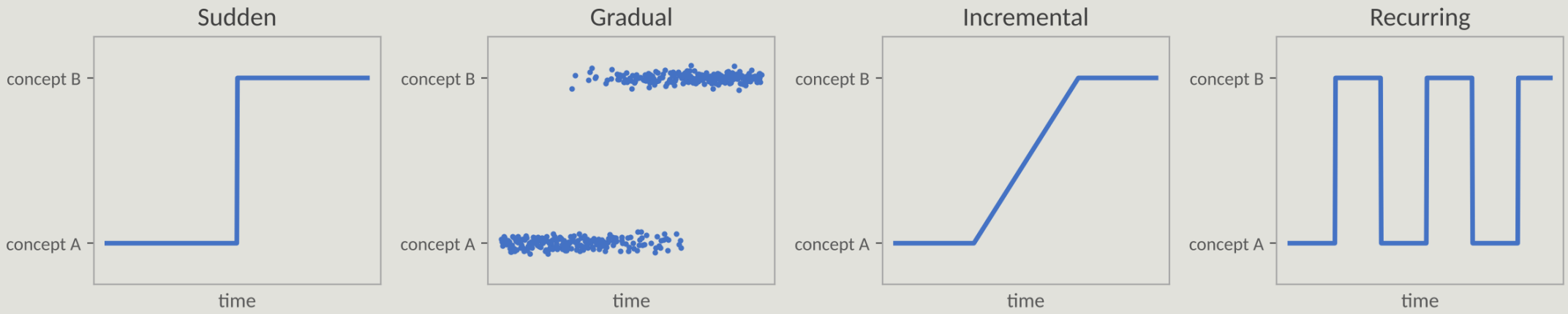
Concept: the relationship between the features and the target, i.e., P(Y|X). Concept drift: the concept changes over time

$$\exists X: \quad P_{t_0}(Y|X) \neq P_{t_1}(Y|X)$$

- **Real drift**: P(Y|X) changes, the decision boundary moves and the model becomes wrong even if the input data look the same
- **Virtual drift**: only P(X) changes (covariate shift); the boundary is still valid, but the model may work in regions poorly covered by the training data



Types of Concept Drift



- Sudden: the concept changes abruptly (a new regulation, the COVID lockdown)
- Gradual: old and new concept alternate before the new one prevails (customers progressively moving to a new product)
- Incremental: the concept changes slowly through intermediate states (sensor ageing, inflation)
- Recurring: old concepts come back, often with a seasonal pattern (summer vs winter gas consumption, Christmas sales)

A single anomalous value is an outlier, not a drift: a drift is a *persistent* change of the concept

Detecting Drift with Labels

When the true labels become available (even with some delay), drift is detected by monitoring the model performance on recent data

- Error rate, accuracy or F1 computed on a sliding window and compared with the values measured at deployment time
- DDM (Drift Detection Method): the errors of the classifier are modeled as a Bernoulli process; p_i is the error rate after i instances, s_i its standard deviation, p_{min} and s_{min} the values observed when $p_i + s_i$ was minimum

$$\text{warning if } p_i + s_i \geq p_{min} + 2\,s_{min} \qquad \text{drift if } p_i + s_i \geq p_{min} + 3\,s_{min} \qquad s_i = \sqrt{p_i(1-p_i)/i}$$

Limitation: in many applications labels arrive late (a credit default after months) or never (unlabeled production data)

Detecting Drift with Labels

When the true labels become available (even with some delay), drift is detected by monitoring the model performance on recent data

- Error rate, accuracy or F1 computed on a sliding window and compared with the values measured at deployment time
- DDM (Drift Detection Method): the errors of the classifier are modeled as a Bernoulli process; p_i is the error rate after i instances, s_i its standard deviation, p_{min} and s_{min} the values observed when $p_i + s_i$ was minimum

$$s_i = \sqrt{p_i(1-p_i)/i}$$

A **Bernoulli process** is a sequence of random trials that satisfy three conditions:
1.Each trial has **only two possible outcomes**, conventionally called success (1) and failure (0).
2.The probability of success p is **constant** across all trials.
3.The trials are **independent**: the outcome of one trial does not affect the outcomes of the others.

In a Bernulli process the standard deviation can be computed as $s_i = \sqrt{p_i(1-p_i)/i}$

ever (unlabeled production

Detecting Drift without Labels

Compare the distribution of a reference window (training data) with a window of recent production data

- Population Stability Index (PSI) measures how much the distribution of a variable has changed between a **reference population** *r* and a **current population** *c*
  1. Divide the variable into *B* bins, using the **same boundaries** for both populations.
  2. Calculate the proportion of observations in each bin.
  3. Compare the proportions:

$$PSI = \sum_{b=1}^{B} (c_b - r_b) \ln \frac{c_b}{r_b}$$

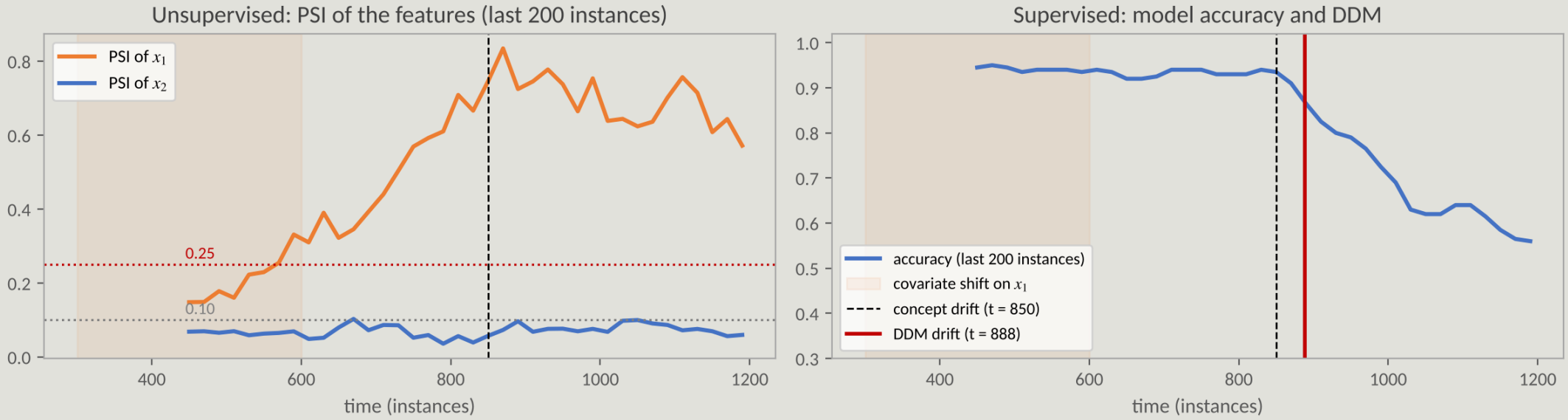
PSI	Interpretation
< 0.10	no significant shift
0.10 – 0.25	moderate shift
> 0.25	significant shift

- **Domain classifier**: train a classifier to distinguish reference from current data; AUC ≈ 0.5 means the two distributions cannot be told apart. Thus a drift takes place when AUC >> 0.5.

PSI see only changes of P(X): a concept drift with unchanged P(X) is invisible without labels

Drift Detection: an Example

Synthetic stream of 1,200 instances with two features; a logistic model is trained on the first 250. Between t = 300 and 600 the mean of x_1 shifts (covariate shift); at t = 850 the boundary rotates (concept drift)



- PSI detects the covariate shift on x_1 (above 0.25 from t = 570), but accuracy does not change: not every shift harms the model
- PSI does not react to the concept drift since P(X) does not change; accuracy drops from 0.94 to about 0.6 and DDM signals the drift at t = 888

Dataset Shift vs Concept Drift: an Intuition

Think of the model as a doctor who learnt to diagnose from the patients of one hospital:

Dataset shift: the patients change

The doctor moves from a pediatric hospital to a geriatric ward: other ages, other diseases are frequent, but the same symptoms still mean the same disease

→ the knowledge is still valid, it is applied to unfamiliar cases

Concept drift: the medicine changes

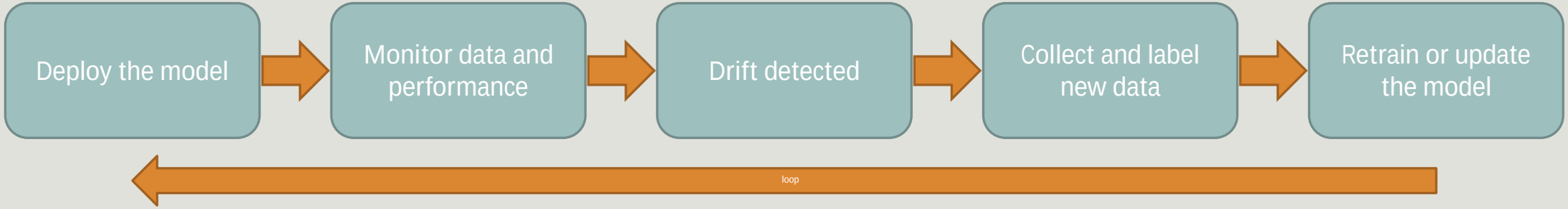
A new virus variant appears: the same symptoms (fever, cough) now point to a different disease, even for patients identical to those seen before

→ the knowledge itself is obsolete

- Dataset shift is the umbrella term: P(X, Y) in deployment differs from training, often because the Sampling Frame does not match the Target Population (*where* the data come from)
- Concept drift is the case in which P(Y|X) changes, typically over time because the phenomenon itself evolves (*when* the data were collected)
- Consequences: with unchanged P(Y|X) the old labels are still correct, so we fix the data (expand, re-weight, recalibrate); with concept drift the old labels are wrong for the new concept: new labels and retraining are needed

Dataset shift: same rules, different data. Concept drift: same data, different rules

Reacting to Drift



- When: periodic retraining (fixed schedule) or triggered retraining (when a detector fires)
- Which data: a sliding window that forgets old data, or weights that favor recent instances; old models can be kept for recurring concepts
- How: full retraining, or online/incremental learning that updates the model instance by instance
- Data-centric: labeling all new data is expensive; label only the most informative new instances → **Active Learning**

Dataset Shift: Summary

	Covariate shift	Label shift	Concept drift
What changes	Distribution of X in the Target Population	Class frequencies in the Target Population	The feature → label relationship P(Y|X)
What is stable	P(Y|X)	P(X|Y)	— (nothing)
Cause	The Sampling Frame does not cover the Target Population	The Target Population has a different class mix	The world has changed after training
Detection	Compare feature distributions (KS test, PSI)	Compare predicted vs actual class frequencies	Monitor model performance on recent labeled data
Remedy	Expand the Sampling Frame; re-weight training examples	Adjust class priors or decision threshold	Retrain on recent data; active data collection

Responding to Dataset Shift

Strategy	When to use
Expand the Sampling Frame	Covariate shift: collect data that better cover the Target Population
Re-weight training examples	Covariate shift: give higher weight to examples similar to the Target Population
Adjust class priors / threshold	Label shift: recalibrate without full retraining when P(X|Y) is unchanged
Continuous retraining	Concept drift: periodically retrain on a sliding window of recent labeled data
Active data collection	Concept drift: identify regions where the model is now wrong and selectively collect new labels there

Connecting to Active Learning: when drift is detected, active learning guides which new examples to label first

Dataset Shift: Key Takeaways

1. **Sampling Frame ≠ Target Population is the root cause of all shift**
   - A model is only as good as the match between where it was trained and where it is deployed
2. **Three distinct mechanisms, three distinct remedies**
   - Covariate shift → re-weight / expand data; label shift → recalibrate priors; concept drift → retrain
3. **Shift is not always visible at training time**
   - It reveals itself through degraded performance in production, hence monitoring is essential
4. **Monitor model performance on recent labeled data**
   - A sustained drop in accuracy, F1 or AUC is the primary signal that drift has occurred
5. **Use feature-distribution tests for early warning**
   - PSI and KS test detect covariate shift before ground-truth labels are available

Active Learning

Unlabeled data are often abundant and cheap, while labels are scarce and expensive: a radiologist annotating CT scans, a lawyer classifying contracts, an expert inspecting defects

Active Learning: the learner chooses which instances should be labeled by an oracle (a human annotator), in order to reach a good performance with the minimum labeling cost

- **Pool-based**: a large pool U of unlabeled data is available; at each iteration the most informative instances of U are selected (the most common scenario)
- **Stream-based selective sampling**: instances arrive one at a time and the learner decides whether to ask for the label or discard them (e.g., production data under drift)
- **Membership query synthesis**: the learner generates new instances to be labeled (rarely used: synthetic instances may be meaningless for a human annotator)

The Active Learning Loop



- Stopping criteria: the labeling budget is exhausted, the performance on a validation set reaches a plateau, or the uncertainty of the remaining instances falls below a threshold
- Batch mode: in practice *k* instances are queried at each iteration to reduce the retraining cost and to parallelize the annotation

Query Strategies: Uncertainty Sampling

Query the instances on which the current model is least confident:

Least confidence

$$x_{LC}^{*} = \arg\max_{x} \; (1 - P(\hat{y} \mid x))$$

Margin

$$x_{M}^{*} = \arg\min_{x} \; (P(\hat{y}_1 \mid x) - P(\hat{y}_2 \mid x))$$

Entropy

$$x_{H}^{*} = \arg\max_{x} \; -\sum_{y} P(y \mid x) \log_2 P(y \mid x)$$

where:

- $\hat{y} = \text{argmax}_y P(y|x)$ is the most likely class
- $\hat{y}_1$, $\hat{y}_2$ are the two most likely classes
- Least confidence queries the instance with the highest 1 – max P(y|x)

Instance	P(y1|x)	P(y2|x)	P(y3|x)	1 – max P(y|x)	Margin	Entropy
xa	0.90	0.05	0.05	0.10	0.85	0.57
xb	0.50	0.45	0.05	0.50	**0.05**	1.23
xc	0.40	0.30	0.30	**0.60**	0.10	**1.57**

The strategies may disagree: least confidence and entropy select **xc**, margin selects **xb** since it looks only at the two most likely classes. With two classes the three strategies coincide.

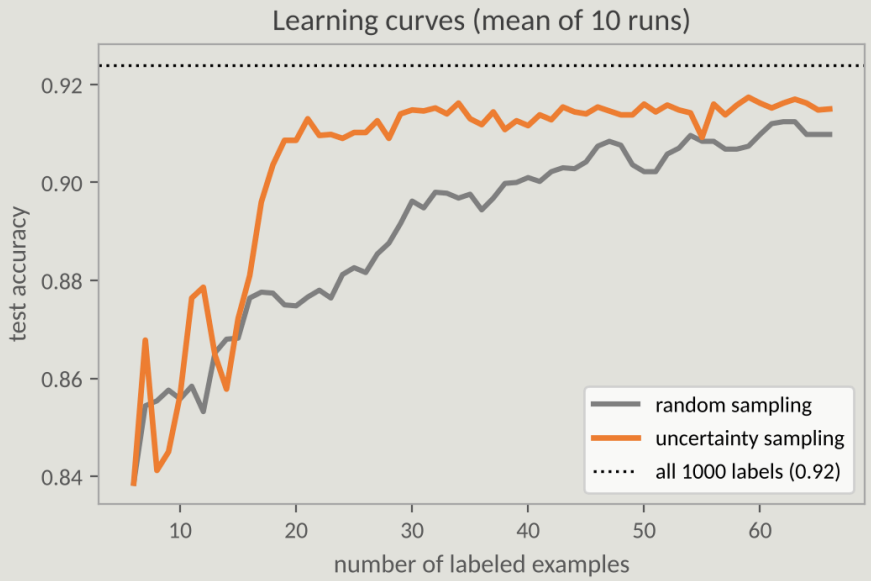
Beyond Uncertainty Sampling

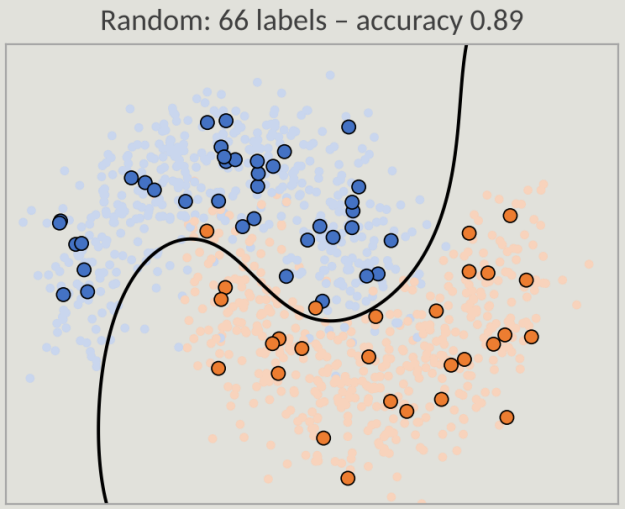
- Query-by-Committee: a committee of models trained on L (e.g., bagging) votes on each instance; query the instances with the maximum disagreement (vote entropy)
- Diversity (representativeness) sampling: select instances that cover the feature space, e.g., one instance per cluster; avoids querying many near-duplicate instances in the same batch
- Expected model change / expected error reduction: query the instance that would change the model (or reduce the expected error) the most; effective but computationally expensive
- Hybrid strategies: uncertainty + diversity, the usual choice in batch mode

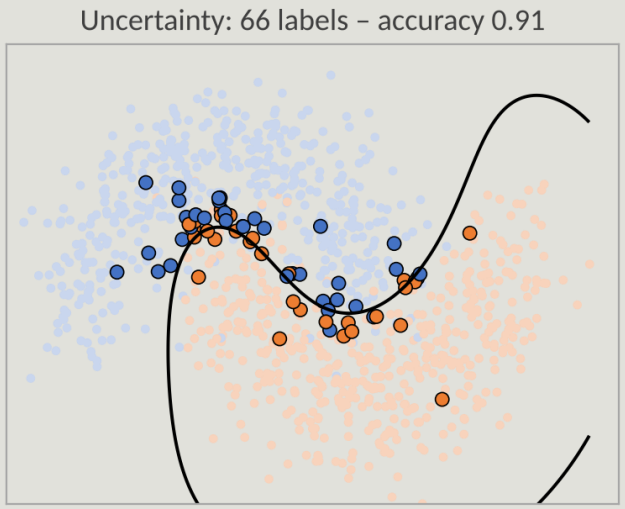
Pure uncertainty sampling tends to select outliers and redundant instances, and produces a labeled set that is not representative of the population (sampling bias)

Active Learning: an Example

Two-moons dataset: pool of 1,000 unlabeled instances, test set of 500. SVM with RBF kernel, 6 initial labels, then one query at a time (uncertainty = smallest distance from the boundary)







- With 20 labels uncertainty sampling reaches 0.91 accuracy (random sampling 0.875); random sampling needs about 60 labels to get the same result
- Queries concentrate along the boundary: far from it there are no labels and the learnt boundary can be wrong (sampling bias)

Data-Centric Sampling Strategies: a Comparison

Sampling Strategy	New information?	Oracle labels?	Main goal
Random / stratified sampling	No	No	Representative (smaller) dataset
Oversampling (SMOTE)	Synthetic, interpolated	No	Rebalance rare classes
Data augmentation	Plausible variations	No	Robustness, more examples
Cost-sensitive learning	No	No	Weight the errors on rare classes
Drift detection + retraining	Yes: new production data	Yes (or no, if unsupervised)	Keep the model aligned with the data
Active learning	Yes: selected real instances	Yes	Best performance with minimum labeling cost

Data-centric AI is not about *more* data, but about the *right* data: representative for evaluation, informative for training, and up to date with the deployment conditions